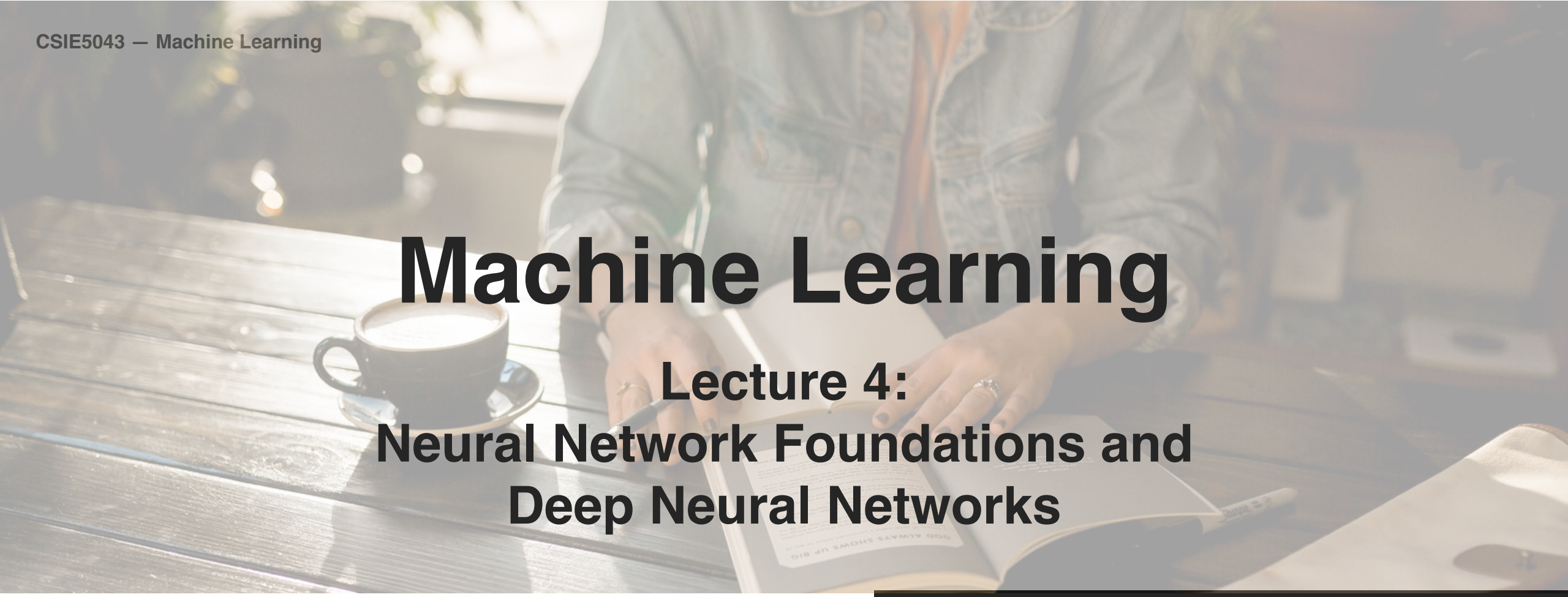


Machine Learning

Lecture 4:
Neural Network Foundations and Deep Neural Networks



國立臺灣大學
National Taiwan University



National Taiwan University

Department of Computer Science and Information Engineering

Prof. Chun-Yi Lee

Outline

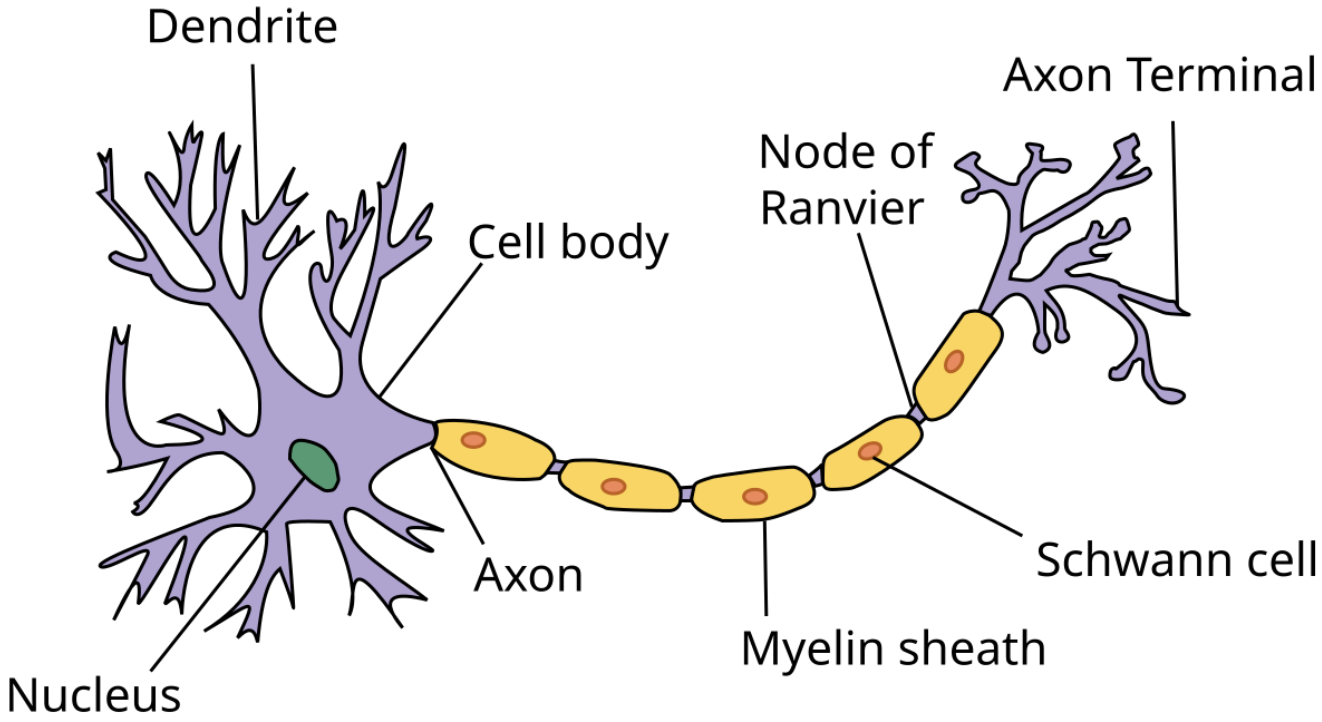
- **Neural Network**
- Backpropagation
- Training scheme
- Deep neural network



Neural Network

Introduction of Neuron - Neuron in the Brain

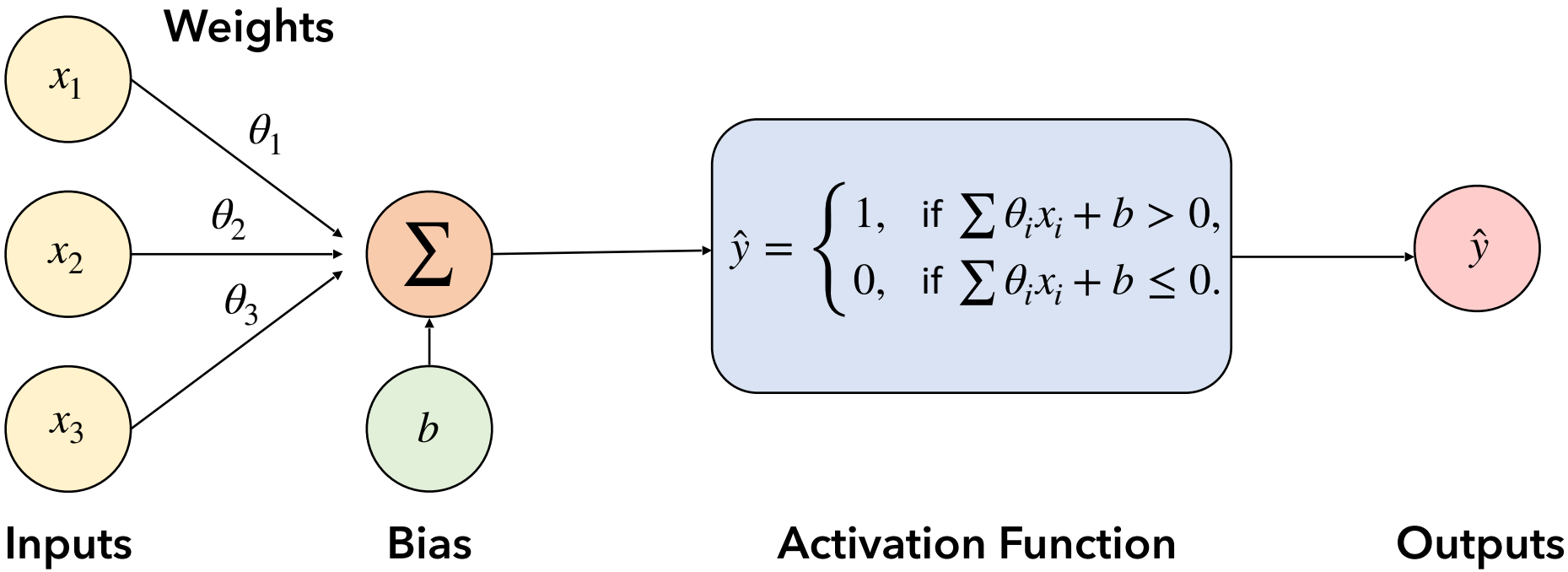
- Biological neurons are the basic units of the brain and nervous system
- They receive signals through dendrites, process them in the cell body, and transmit outputs via the axon
- Connections between neurons, called synapses, enable communication and complex brain functions



Neural Network

Recall: Perceptron - The Simplest Neural Unit

- Initial proposal of connectionist networks
- It is a binary classifier that makes predictions based on a linear combination of input features
- The original perceptron is a **single-layer model** and does not have any hidden layers



Neural Network

Single-Layer Perceptron - Limitations

What single-layer perceptron **CAN** do …

- Solve **linearly separable** problems
- Perform binary classification
- Represent decision boundaries as a line/ hyperplane
- Capture simple linear relationships between input and output

Examples:

- AND gate
- OR gate

What single-layer perceptron **CANNOT** do …

- Solve **non-linearly separable** problems
- Handle complex decision boundaries
- Learn non-linear patterns or higher-order features
- Go beyond a single straight-line separation

Examples:

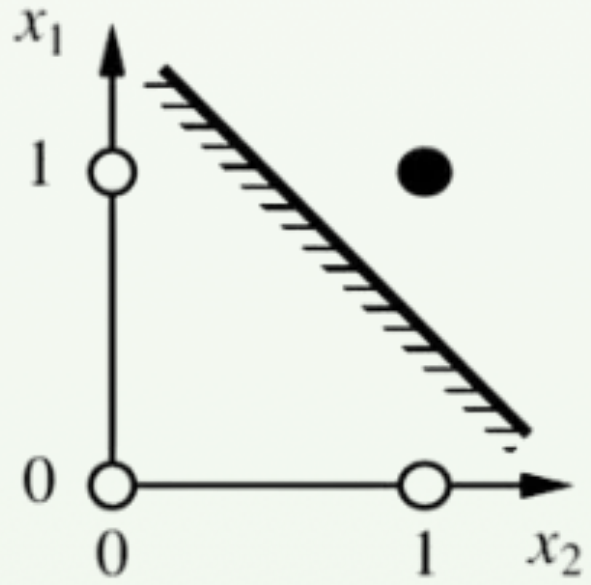
- XOR gate

Neural Network

Single-Layer Perceptron - AND Gate

Truth Table - AND		
x_1	x_2	Output
0	0	0
0	1	0
1	0	0
1	1	0

Linearly Separable



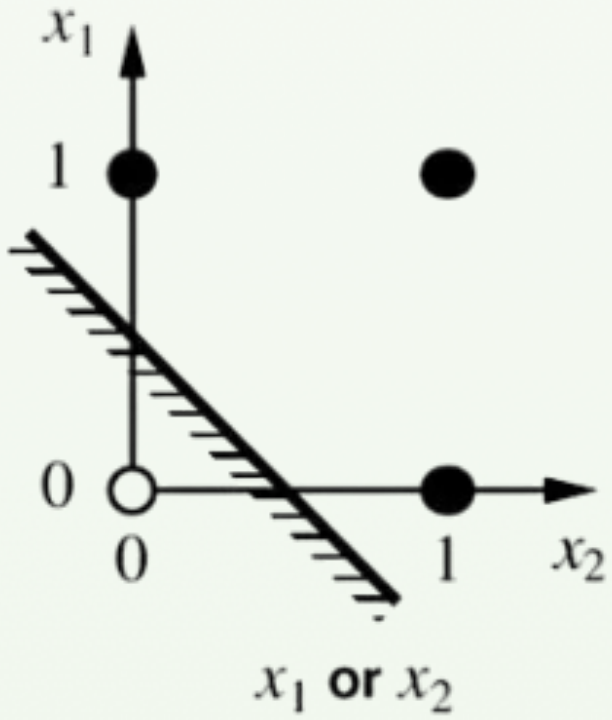
x_1 **and** x_2

Neural Network

Single-Layer Perceptron - OR Gate

Truth Table – OR		
x_1	x_2	Output
0	0	0
0	1	1
1	0	1
1	1	1

Linearly Separable

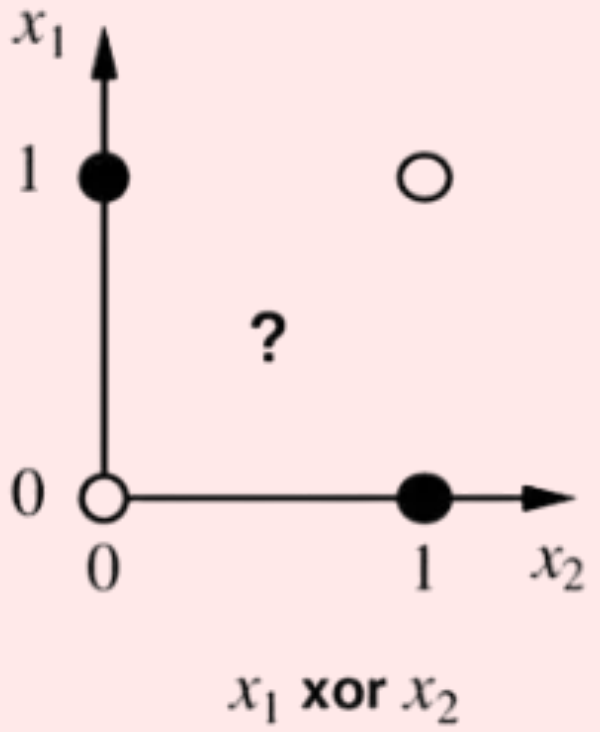


Neural Network

Single-Layer Perceptron - XOR Gate

Truth Table - XOR		
x_1	x_2	Output
0	0	0
0	1	1
1	0	1
1	1	0

Non-Linearly Separable



x_1 **xor** x_2

Neural Network

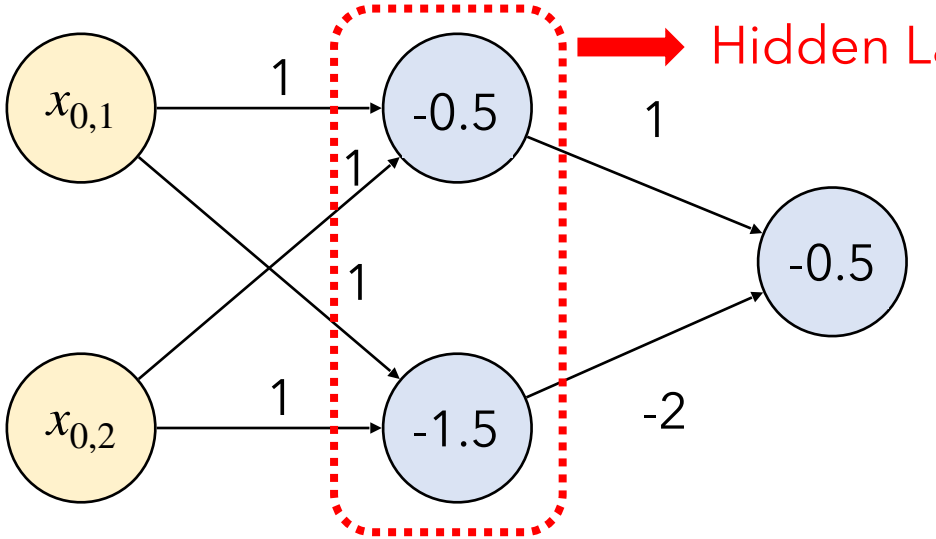
Multi-Layer Perceptron - Solution for Non-Linearly Separable Problems

Use a two-layer perceptron to solve the XOR gate problem:

- Two inputs x_1, x_2 and one output $\hat{y}$
- One **hidden layer**



$$x_{1,1} = f(g_{1,1}(x)) = f(z_{1,1})$$
$$g_{1,1}(x) = z_{1,1} = 1 \cdot x_{0,1} + 1 \cdot x_{0,2} - 0.5$$



$$\hat{y} = f(g_{2,1}(x)) = f(x_{1,1} - 2x_{1,2} - 0.5)$$
$$g_{2,1}(x) = 1 \cdot x_{1,1} + (-2) \cdot x_{1,2} - 0.5$$

$$f(x) = \begin{cases} 1, & \text{if } x > 0, \\ 0, & \text{if } x \leq 0. \end{cases}$$

$$x_{1,2} = f(g_{1,2}(x)) = f(z_{1,2})$$
$$g_{1,2}(x) = z_{1,2} = 1 \cdot x_{0,1} + 1 \cdot x_{0,2} - 1.5$$

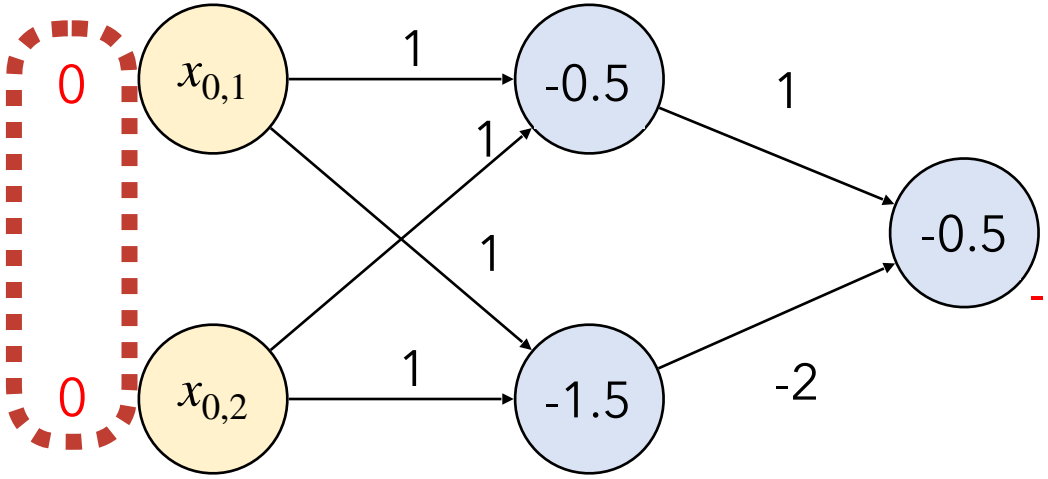
Neural Network

Multi-Layer Perceptron - Part (1/4)

Use a two-layer perceptron to solve XOR gate problem:

- Two inputs x_1, x_2 and one output $\hat{y}$
- One hidden layer

Truth Table - XOR		
$x_{0,1}$	$x_{0,2}$	Output
0	0	0
0	1	1
1	0	1
1	1	0



0 $x_{1,1} = f(g_{1,1}(x)) = f(z_{1,1})$

-0.5 $g_{1,1}(x) = z_{1,1} = 1 \cdot x_{0,1} + 1 \cdot x_{0,2} - 0.5$

0 $x_{1,2} = f(g_{1,2}(x)) = f(z_{1,2})$

-1.5 $g_{1,2}(x) = z_{1,2} = 1 \cdot x_{0,1} + 1 \cdot x_{0,2} - 1.5$

0 $\hat{y} = f(g_{2,1}(x)) = f(x_{1,1} - 2x_{1,2} - 0.5)$

-0.5 $g_{2,1}(x) = 1 \cdot x_{1,1} + (-2) \cdot x_{1,2} - 0.5$

$$f(x) = \begin{cases} 1, & \text{if } x > 0, \\ 0, & \text{if } x \leq 0. \end{cases}$$

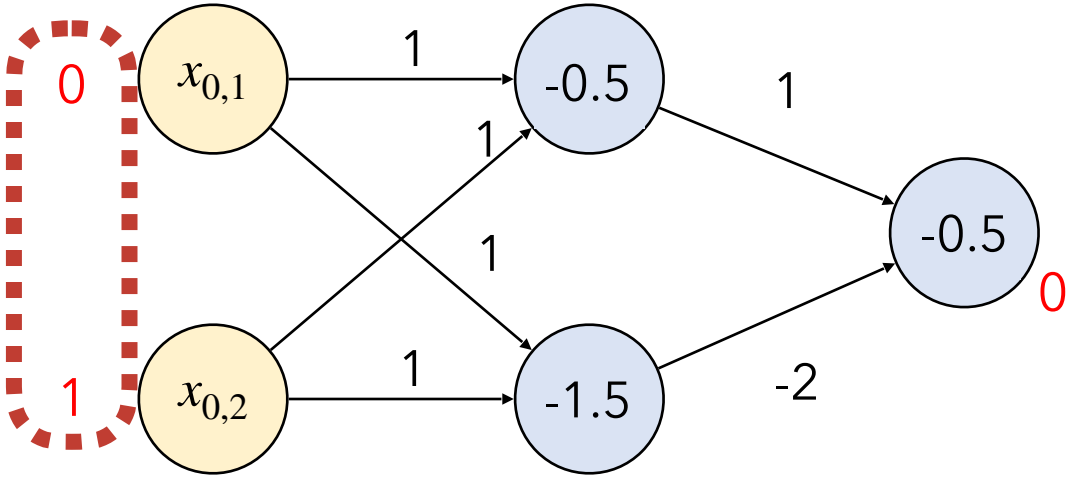
Neural Network

Multi-Layer Perceptron - Part (2/4)

Use a two-layer perceptron to solve XOR gate problem:

- Two inputs x_1, x_2 and one output $\hat{y}$
- One hidden layer

Truth Table - XOR		
$x_{0,1}$	$x_{0,2}$	Output
0	0	0
0	1	1
1	0	1
1	1	0



1 $x_{1,1} = f(g_{1,1}(x)) = f(z_{1,1})$

0.5 $g_{1,1}(x) = z_{1,1} = 1 \cdot x_{0,1} + 1 \cdot x_{0,2} - 0.5$

1 $\hat{y} = f(g_{2,1}(x)) = f(x_{1,1} - 2x_{1,2} - 0.5)$

0.5 $g_{2,1}(x) = 1 \cdot x_{1,1} + (-2) \cdot x_{1,2} - 0.5$

0 $x_{1,2} = f(g_{1,2}(x)) = f(z_{1,2})$

-0.5 $g_{1,2}(x) = z_{1,2} = 1 \cdot x_{0,1} + 1 \cdot x_{0,2} - 1.5$

$$f(x) = \begin{cases} 1, & \text{if } x > 0, \\ 0, & \text{if } x \leq 0. \end{cases}$$

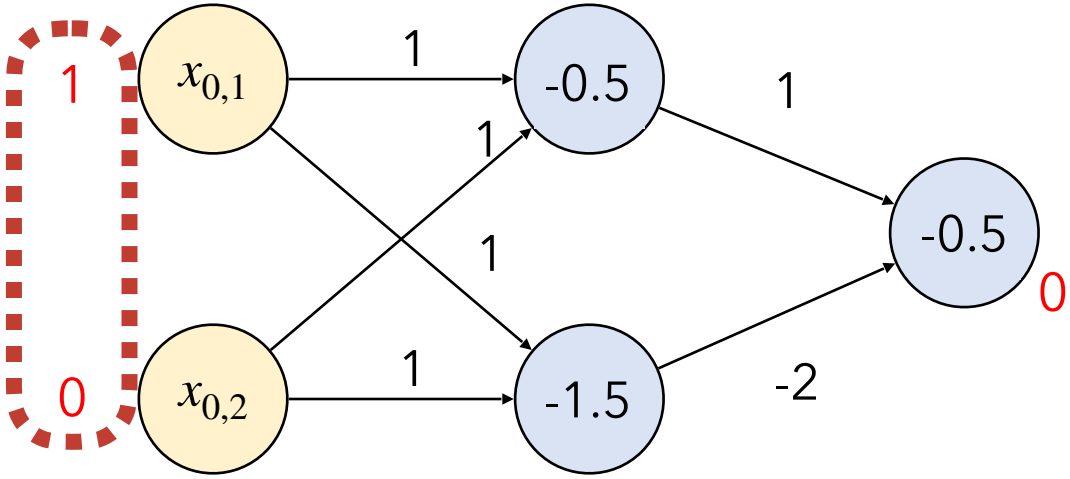
Neural Network

Multi-Layer Perceptron - Part (3/4)

Use a two-layer perceptron to solve XOR gate problem:

- Two inputs x_1, x_2 and one output $\hat{y}$
- One hidden layer

Truth Table - XOR		
$x_{0,1}$	$x_{0,2}$	Output
0	0	0
0	1	1
1	0	1
1	1	0



1 $x_{1,1} = f(g_{1,1}(x)) = f(z_{1,1})$

0.5 $g_{1,1}(x) = z_{1,1} = 1 \cdot x_{0,1} + 1 \cdot x_{0,2} - 0.5$

1 $\hat{y} = f(g_{2,1}(x)) = f(x_{1,1} - 2x_{1,2} - 0.5)$

0.5 $g_{2,1}(x) = 1 \cdot x_{1,1} + (-2) \cdot x_{1,2} - 0.5$

0 $x_{1,2} = f(g_{1,2}(x)) = f(z_{1,2})$

-0.5 $g_{1,2}(x) = z_{1,2} = 1 \cdot x_{0,1} + 1 \cdot x_{0,2} - 1.5$

$$f(x) = \begin{cases} 1, & \text{if } x > 0, \\ 0, & \text{if } x \leq 0. \end{cases}$$

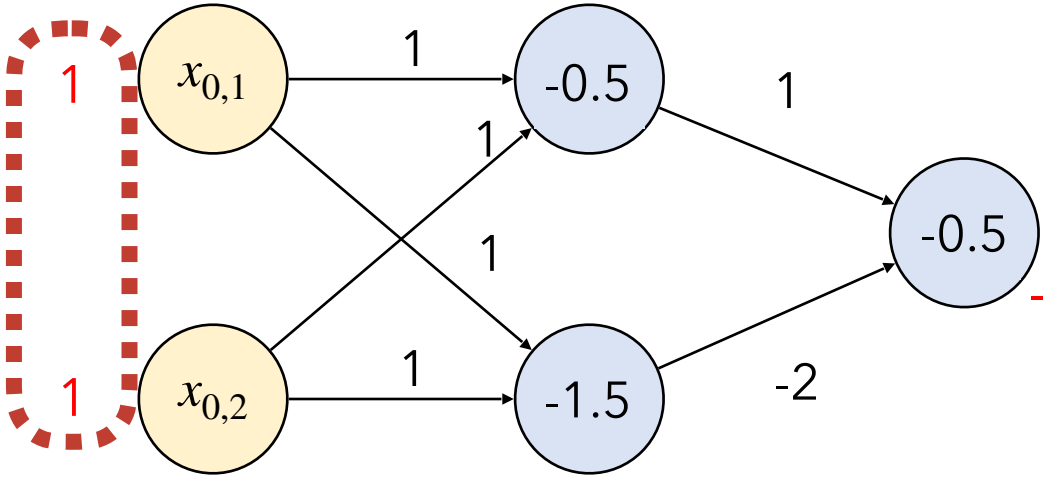
Neural Network

Multi-Layer Perceptron - Part (4/4)

Use a two-layer perceptron to solve XOR gate problem:

- Two inputs x_1, x_2 and one output $\hat{y}$
- One hidden layer

Truth Table - XOR		
$x_{0,1}$	$x_{0,2}$	Output
0	0	0
0	1	1
1	0	1
1	1	0



1 $x_{1,1} = f(g_{1,1}(x)) = f(z_{1,1})$

1.5 $g_{1,1}(x) = z_{1,1} = 1 \cdot x_{0,1} + 1 \cdot x_{0,2} - 0.5$

0 $\hat{y} = f(g_{2,1}(x)) = f(x_{1,1} - 2x_{1,2} - 0.5)$

-1.5 $g_{2,1}(x) = 1 \cdot x_{1,1} + (-2) \cdot x_{1,2} - 0.5$

1 $x_{1,2} = f(g_{1,2}(x)) = f(z_{1,2})$

0.5 $g_{1,2}(x) = z_{1,2} = 1 \cdot x_{0,1} + 1 \cdot x_{0,2} - 1.5$

$$f(x) = \begin{cases} 1, & \text{if } x > 0, \\ 0, & \text{if } x \leq 0. \end{cases}$$

Neural Network

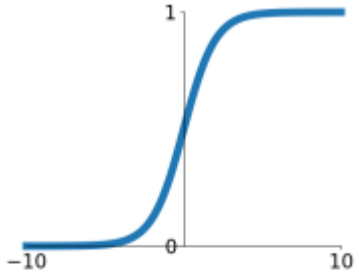
Non-Linear Transformation

Q: A neural network with no activation function?
➞ **Linear classifier!**

Goal: Convert linear input signals from perceptron to a linear/non-linear output signal

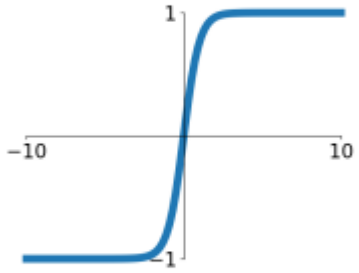
Sigmoid

$$\frac{1}{1+e^{-x}}$$



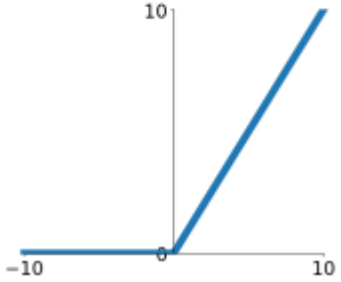
Tanh

$\tanh(x)$



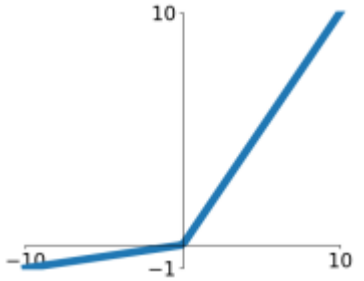
ReLU

$\max(0,\ x)$



Leaky ReLU

$\max(0.1x,\ x)$

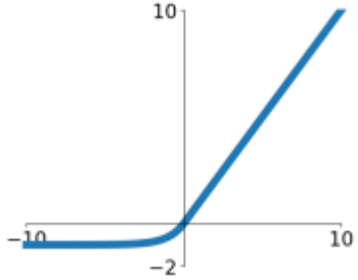


Maxout

$\max(\theta_1^T x + b_1,\ \theta_2^T x + b_2)$

ELU

$$\begin{cases} x, & \text{if } x \geq 0, \\ \alpha\left(e^x - 1\right), & \text{if } x < 0, \end{cases}$$

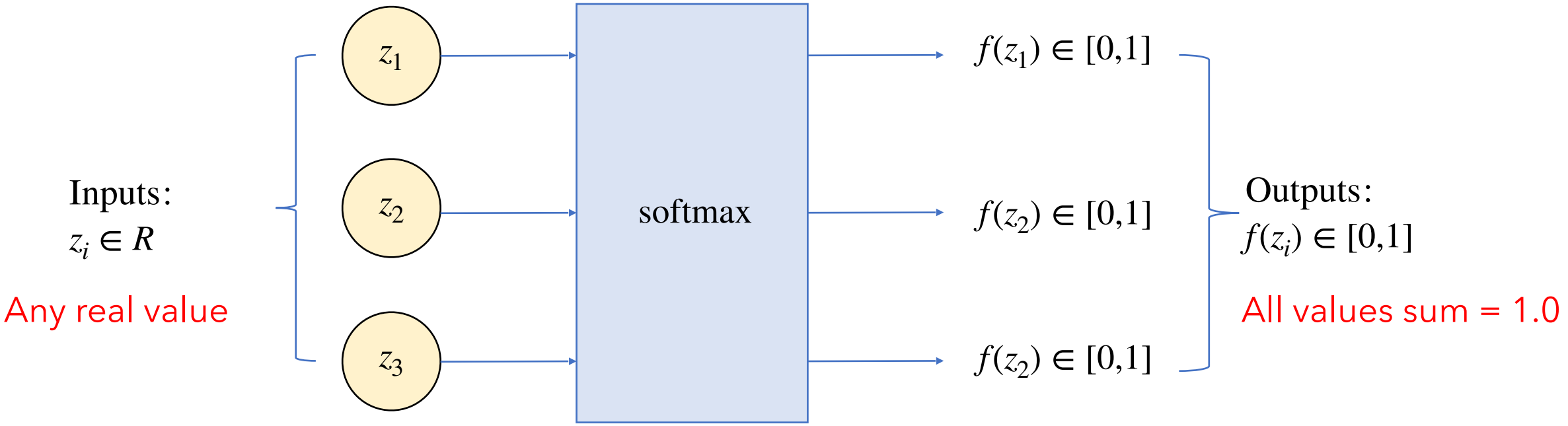


Neural Network

Activation Function - Softmax

Softmax is a function that converts raw scores (**logits**) into a **probability distribution** over classes

$$f(z_i) = \sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{k} e^{z_j}},\ i = 1,2,...,K$$



Outline

- Neural Network
- **Backpropagation**
- Training scheme
- Deep neural network



Backpropagation

Efficient Application of Computing Gradient

Update Rule:

$\theta_{t+1} \leftarrow \theta_t - \eta \nabla L(\theta_t)$, $\eta =$ learning rate, $\nabla L(\theta_t)$ = gradient (direction of steepest ascent)

- For a neural network, we have weights and biases: $\theta = \{w_1, w_2, \ldots, b_1, b_2, \ldots\}$
- To use gradient descent, we need to compute the following:

Compute $\nabla L(\theta_0)$ $\quad \theta_1 \leftarrow \theta_0 - \eta \nabla L(\theta_0)$

Compute $\nabla L(\theta_1)$ $\quad \theta_2 \leftarrow \theta_1 - \eta \nabla L(\theta_1)$

⋮

Millions of parameters, …

To compute the gradients efficiently,
We use ***backpropagation***

Backpropagation

The Idea behind Backpropagation - Chain Rule

Example 1:

$$y = f(u) \; and \; u = g(x)$$

$$\frac{dy}{dx} = \frac{dy}{du} \bullet \frac{du}{dx} = f'\big(g(x)\big) \bullet g'(x)$$

Example 2:

$$x = g(s) \quad y = h(s) \quad z = k(x, y)$$

$$\frac{dz}{dx} = \frac{dz}{dx} \bullet \frac{dx}{ds} + \frac{dz}{dy} \bullet \frac{dy}{ds} = k'\big(g(s)\big) \bullet g'(s) + k'\big(h(s)\big) \bullet h'(s)$$

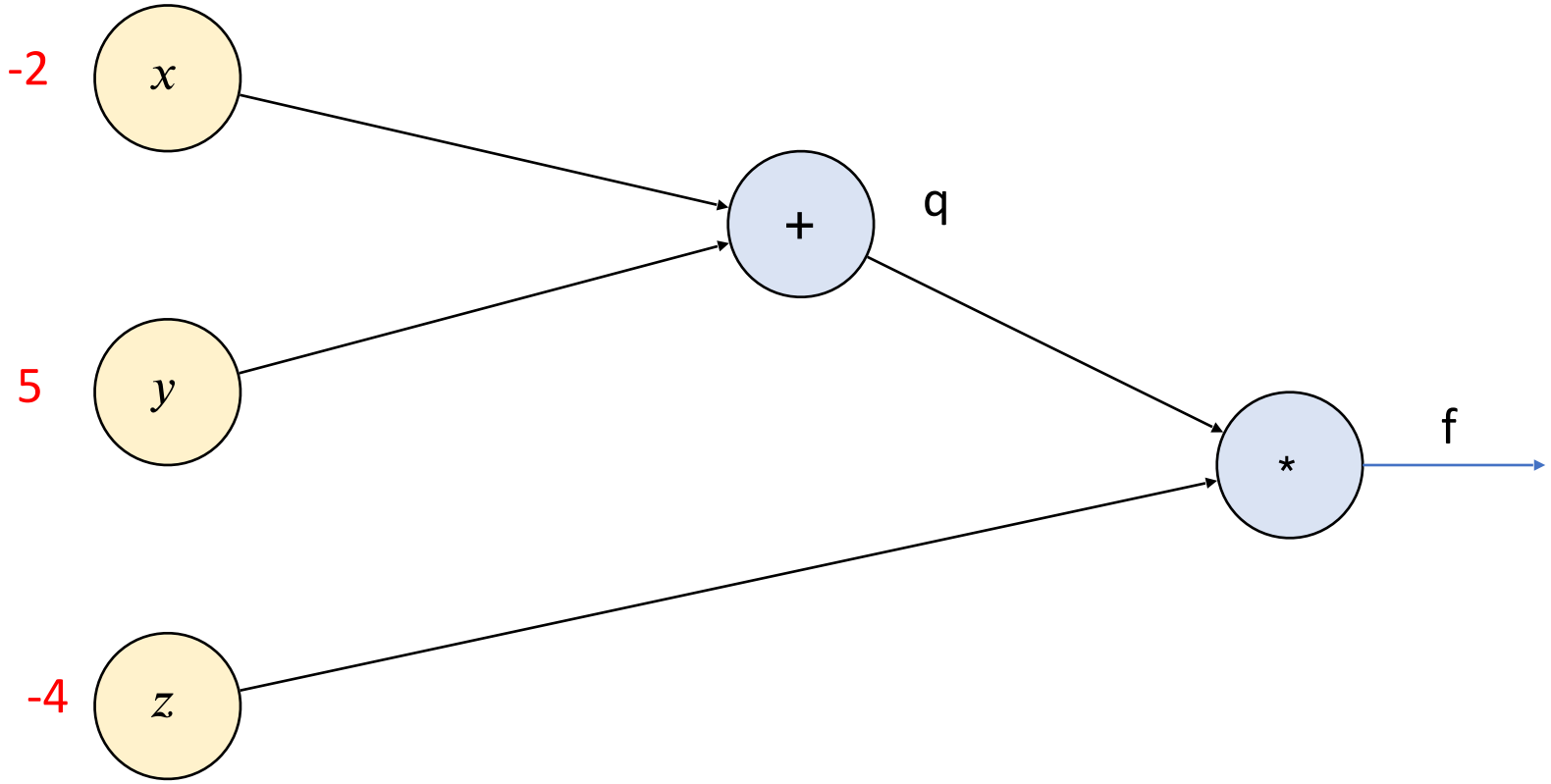
We do not know what the hidden units ought to do, but we can compute how fast the error changes as we change a hidden activity

Backpropagation

Simple Example 1

Example 1 (w/o activation function):

$$f(x, y, z) = (x + y) * z$$

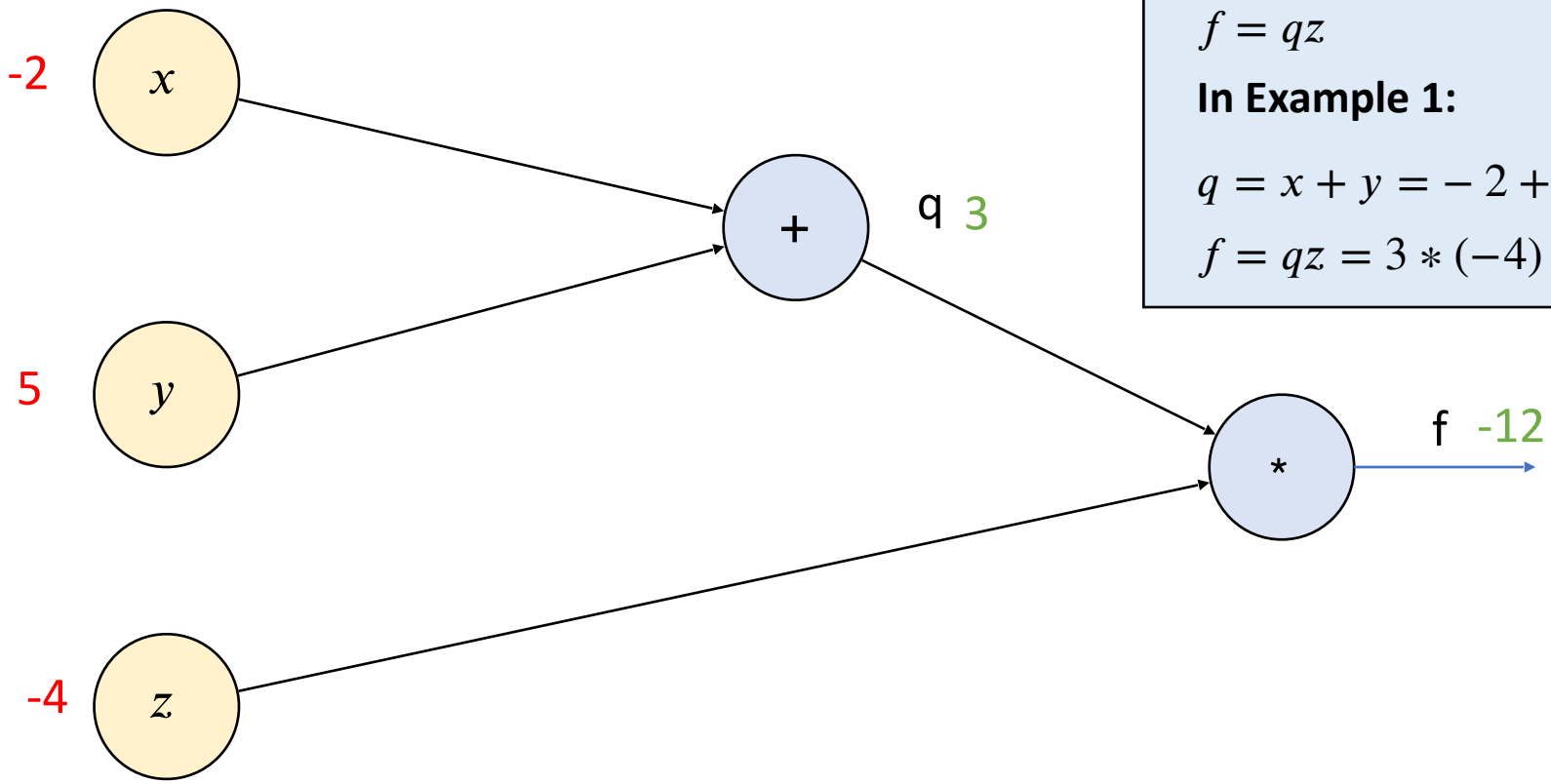


Backpropagation

Simple Example 1 - Forward Pass

Example 1 (w/o activation function):

$$f(x, y, z) = (x + y) * z$$



Forward pass: Compute outputs

$$q = x + y$$

$$f = qz$$

In Example 1:

$$q = x + y = -2 + 5 = 3$$

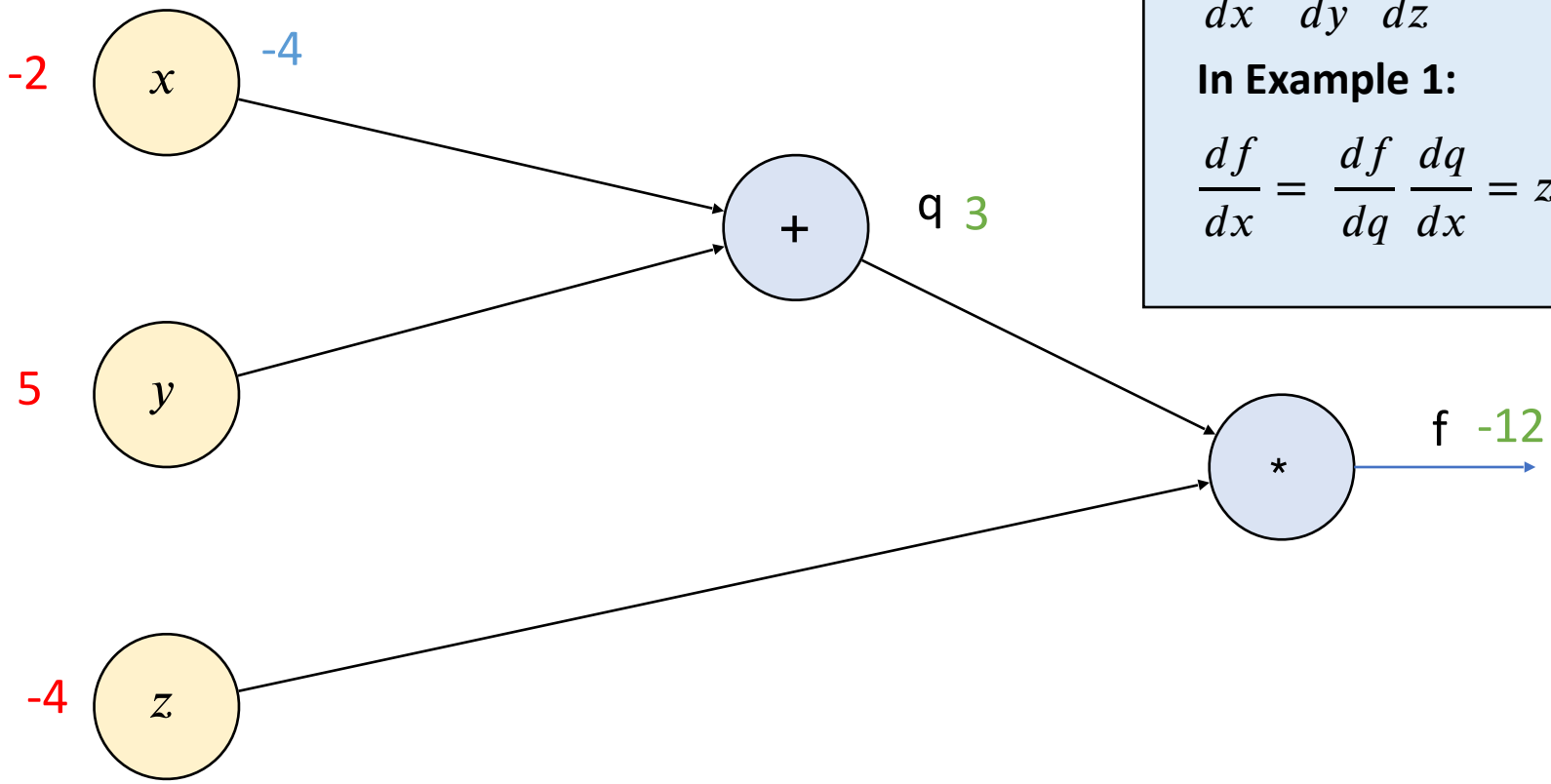
$$f = qz = 3 * (-4) = -12$$

Backpropagation

Simple Example 1 - Backward Pass Part (1/3)

Example 1 (w/o activation function):

$$f(x, y, z) = (x + y) * z$$



Backward pass: Compute derivatives

$$\frac{df}{dx}, \frac{df}{dy}, \frac{df}{dz}$$

In Example 1:

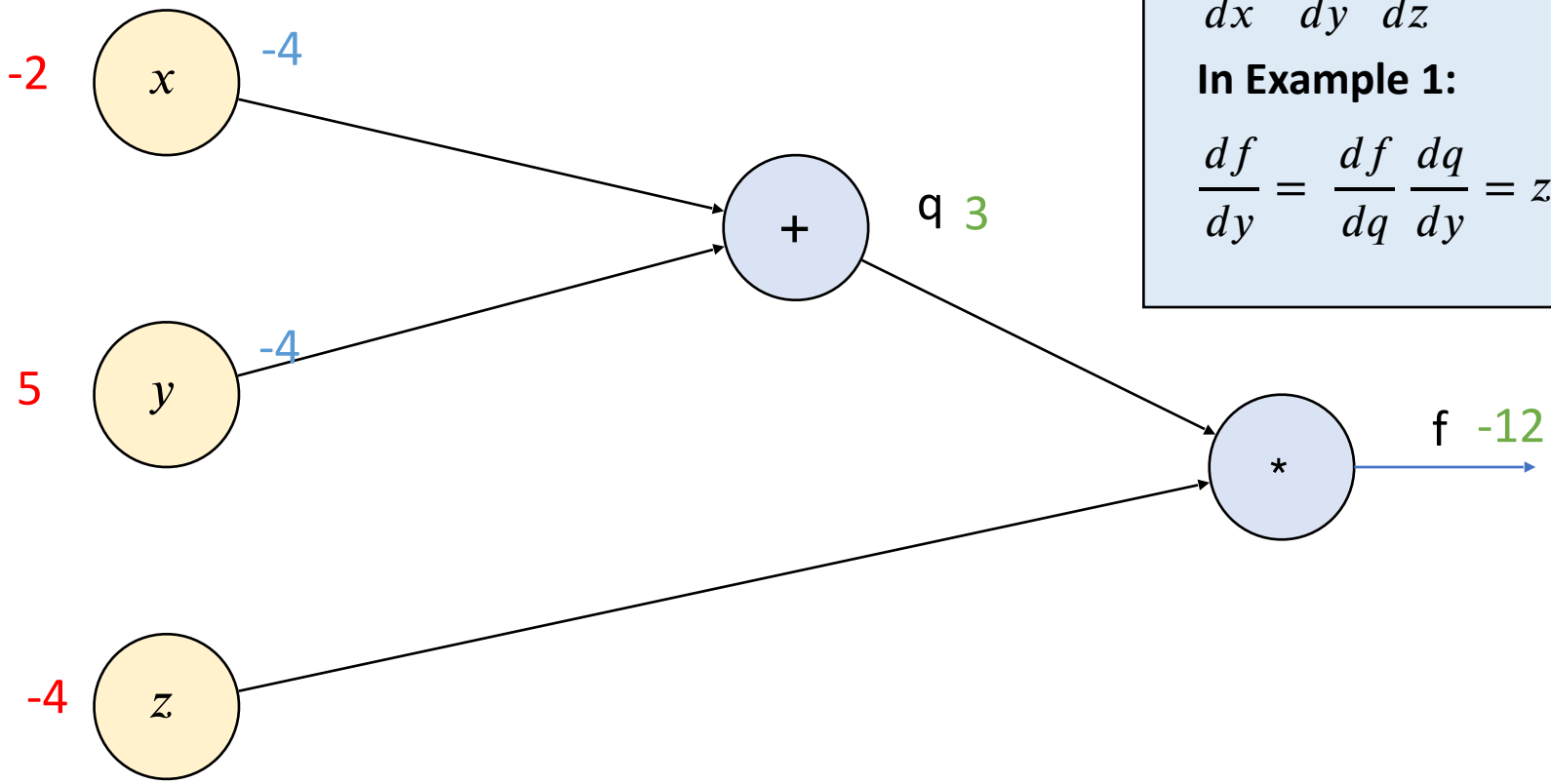
$$\frac{df}{dx} = \frac{df}{dq}\frac{dq}{dx} = z * 1 = -4$$

Backpropagation

Simple Example 1 - Backward Pass Part (2/3)

Example 1 (w/o activation function):

$$f(x, y, z) = (x + y) * z$$



Backward pass: Compute derivatives

$$\frac{df}{dx}, \frac{df}{dy}, \frac{df}{dz}$$

In Example 1:

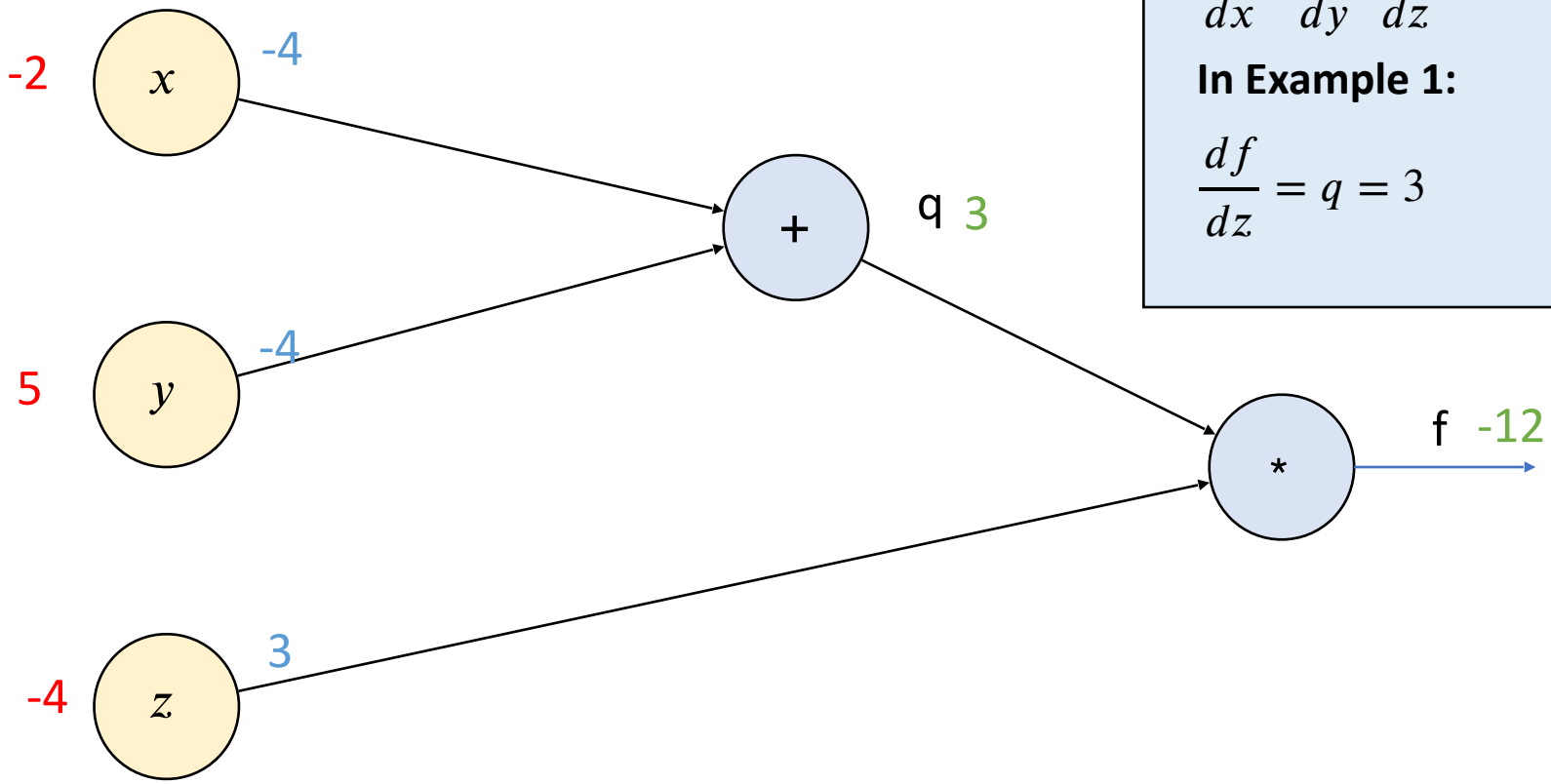
$$\frac{df}{dy} = \frac{df}{dq}\frac{dq}{dy} = z * 1 = -4$$

Backpropagation

Simple Example 1 - Backward Pass Part (3/3)

Example 1 (w/o activation function):

$$f(x, y, z) = (x + y) * z$$



Backward pass: Compute derivatives

$$\frac{df}{dx}, \frac{df}{dy}, \frac{df}{dz}$$

In Example 1:

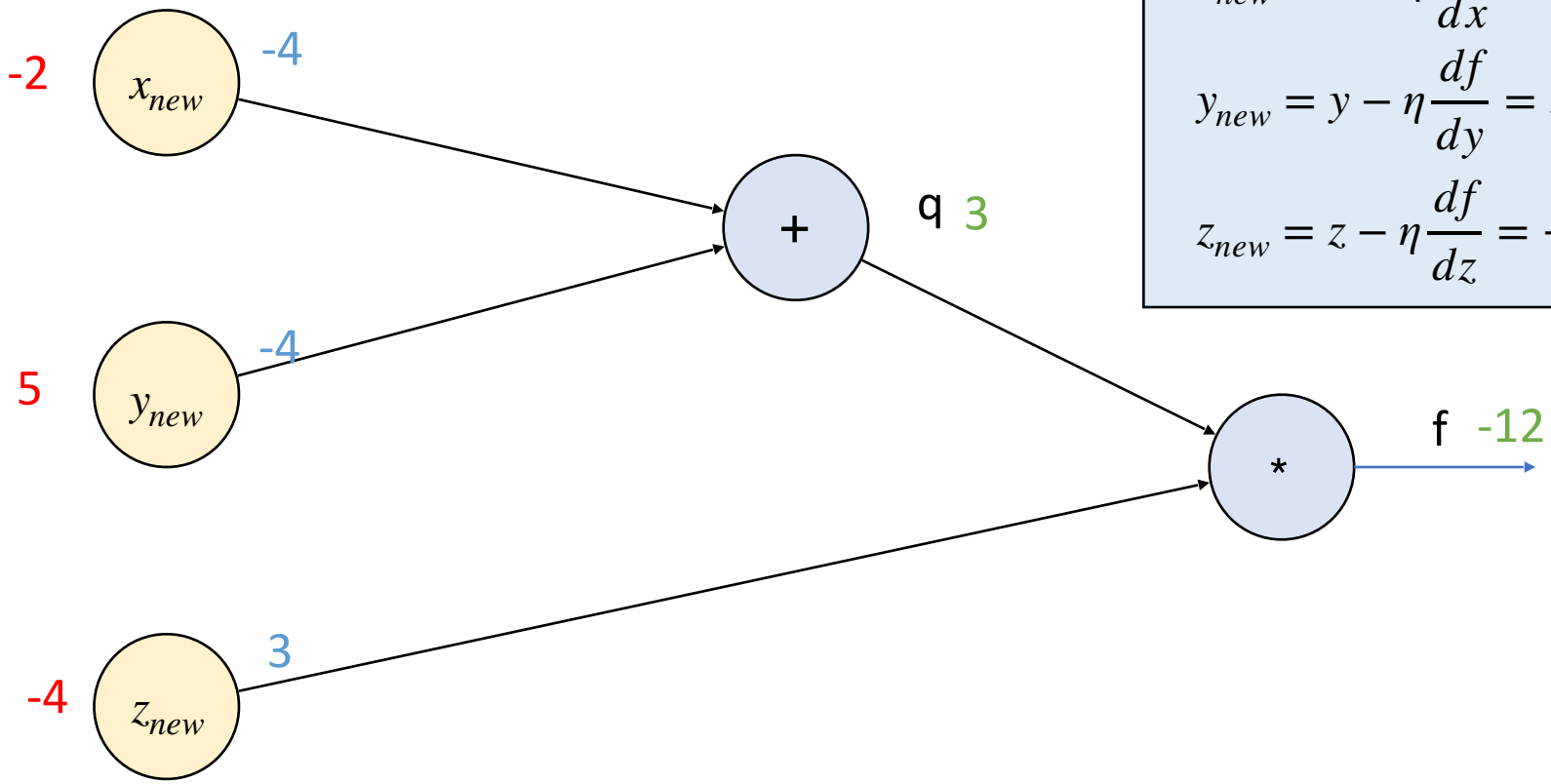
$$\frac{df}{dz} = q = 3$$

Backpropagation

Simple Example 1 - Update Parameters

Example 1 (w/o activation function):

$$f(x, y, z) = (x + y) * z$$



Update Parameters:

$$x_{new} = x - \eta \frac{df}{dx} = -2 - \eta(-4)$$

$$y_{new} = y - \eta \frac{df}{dy} = 5 - \eta(-4)$$

$$z_{new} = z - \eta \frac{df}{dz} = -4 - \eta(3)$$

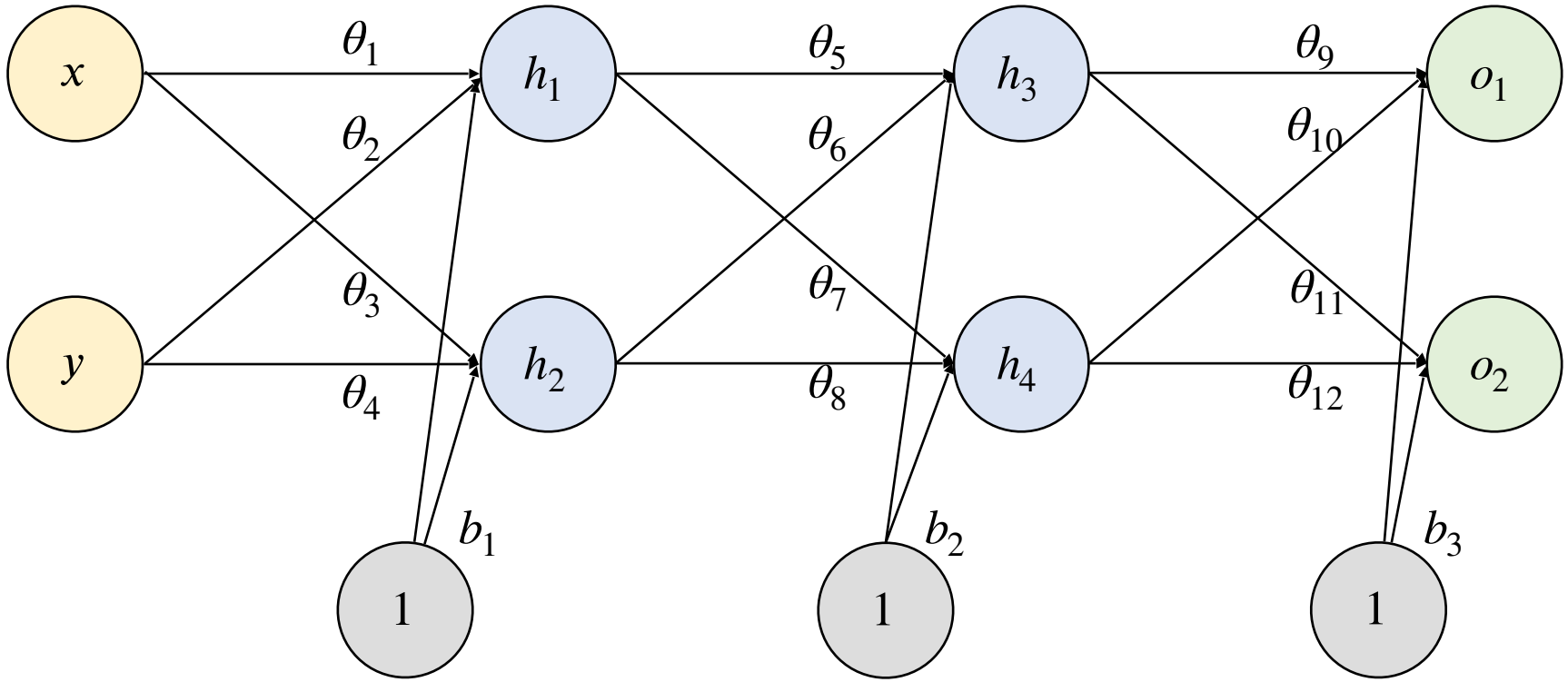
Backpropagation

Simple Example 2

Example 2 (w/ activation function):

Activation Function: Sigmoid

Learning rate: 0.5 **Loss:** L2



Backpropagation

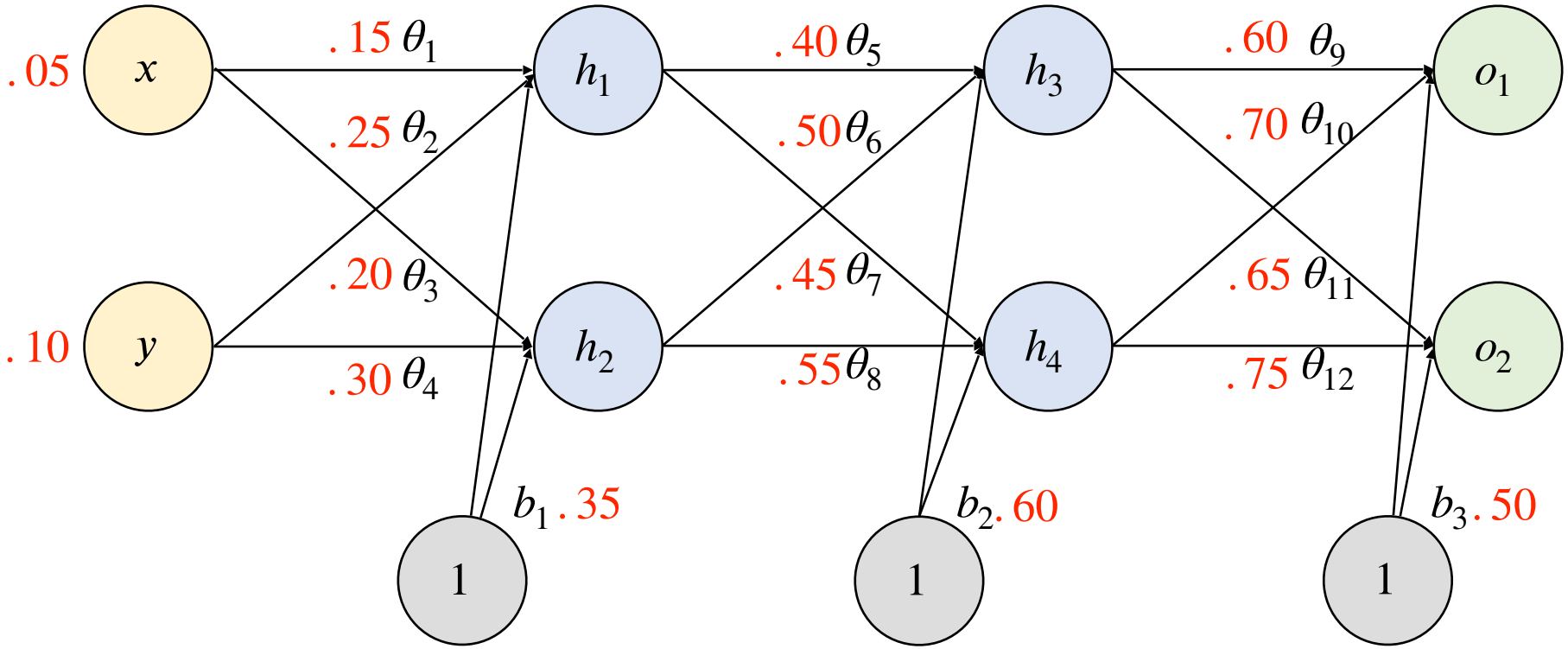
Simple Example 2

Example 2:

Activation Function: Sigmoid

Learning rate: 0.5 **Loss:** L2

Target: $(t1, t2) = (0.01, 0.99)$



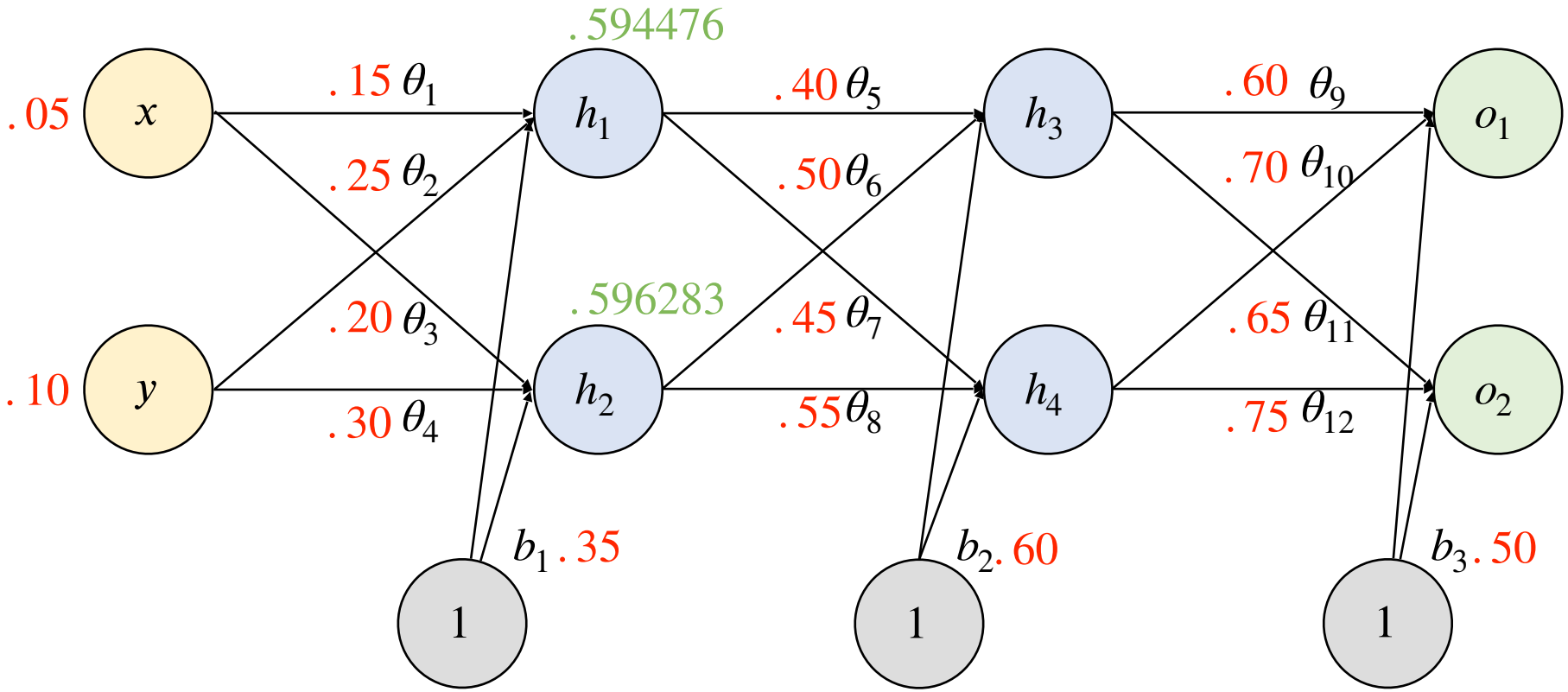
Backpropagation

Simple Example 2 - Forward Pass Part (1/3)

Example 2:

$$z_{h_1} = \theta_1 \cdot x + \theta_2 \cdot y + b_1 = 0.3825 \,, a_{h_1} = \sigma(z_{h_1}) = 0.594476$$

$$z_{h_2} = \theta_3 \cdot x + \theta_4 \cdot y + b_1 = 0.3900 \,, a_{h_2} = \sigma(z_{h_2}) = 0.596283$$

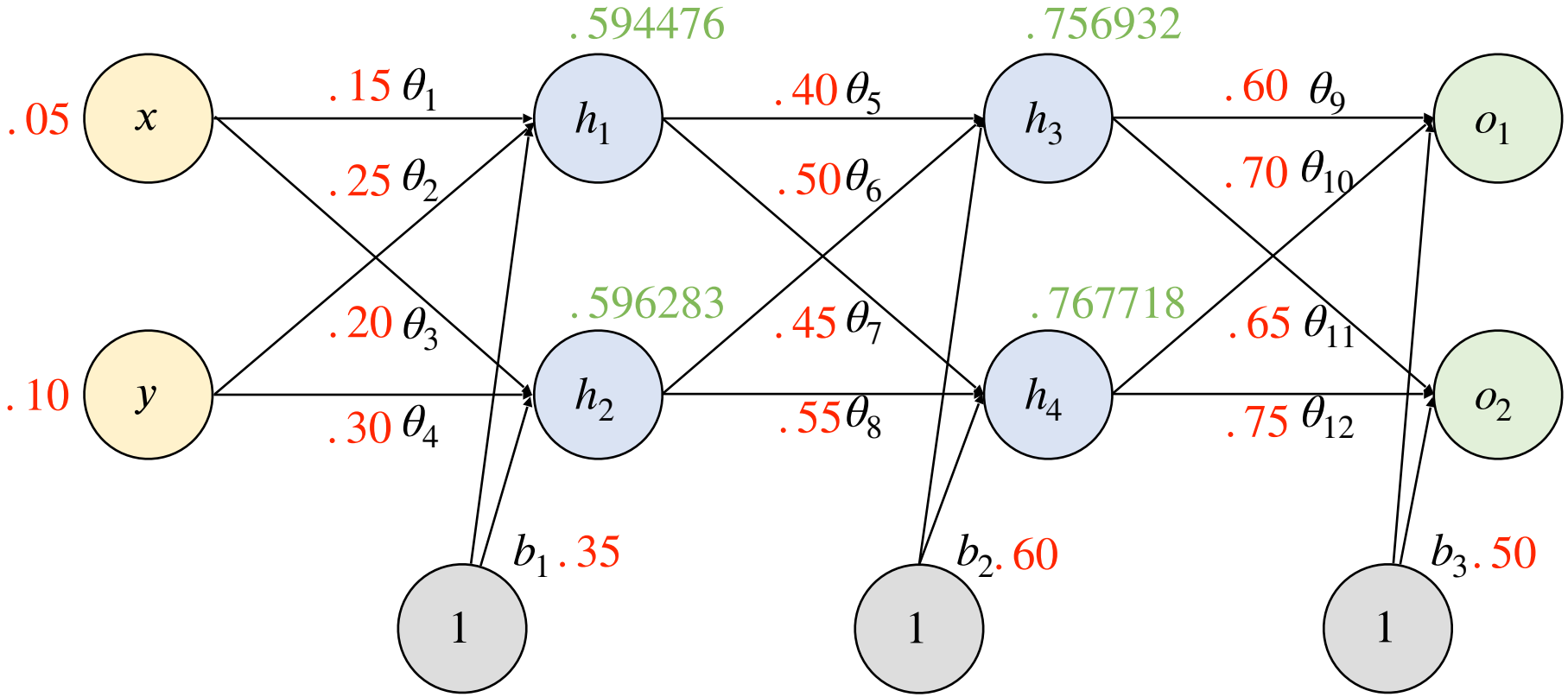


Backpropagation

Simple Example 2 - Forward Pass Part (2/3)

Example 2:

$$z_{h_3} = \theta_5 \cdot x + \theta_6 \cdot y + b_2 = 1.135932 \ , a_{h_3} = \sigma(z_{h_3}) = 0.756932$$
$$z_{h_4} = \theta_7 \cdot x + \theta_8 \cdot y + b_2 = 1.195470 \ , a_{h_4} = \sigma(z_{h_4}) = 0.767718$$

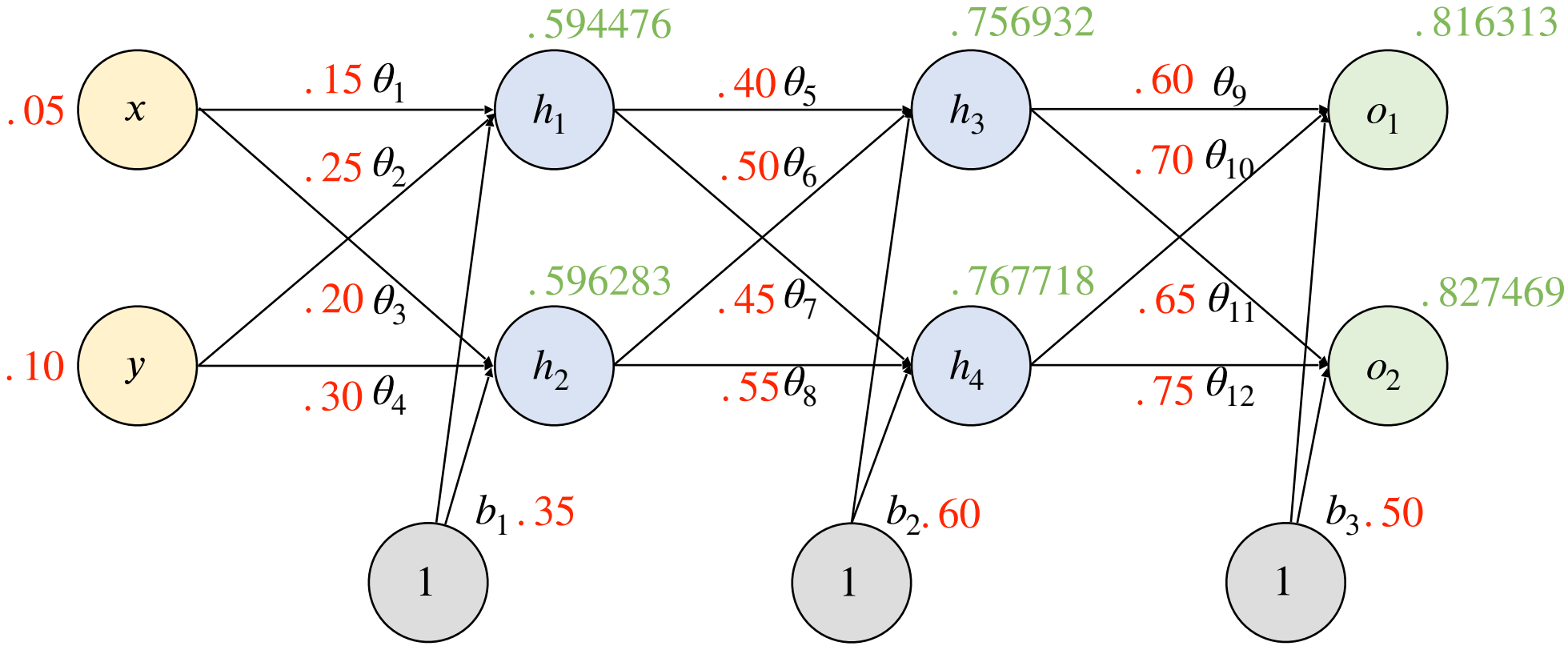


Backpropagation

Simple Example 2 - Forward Pass Part (3/3)

Example 2:

$$z_{o_1} = \theta_9 \cdot a_{h_3} + \theta_{10} \cdot a_{h_4} + b_3 = 1.491562 \ , o_1 = \sigma(z_{o_1}) = 0.816313$$
$$z_{o_2} = \theta_{11} \cdot a_{h_3} + \theta_{12} \cdot a_{h_4} + b_3 = 1.1567794 \ , o_2 = \sigma(z_{o_2}) = 0.827469$$



Backpropagation

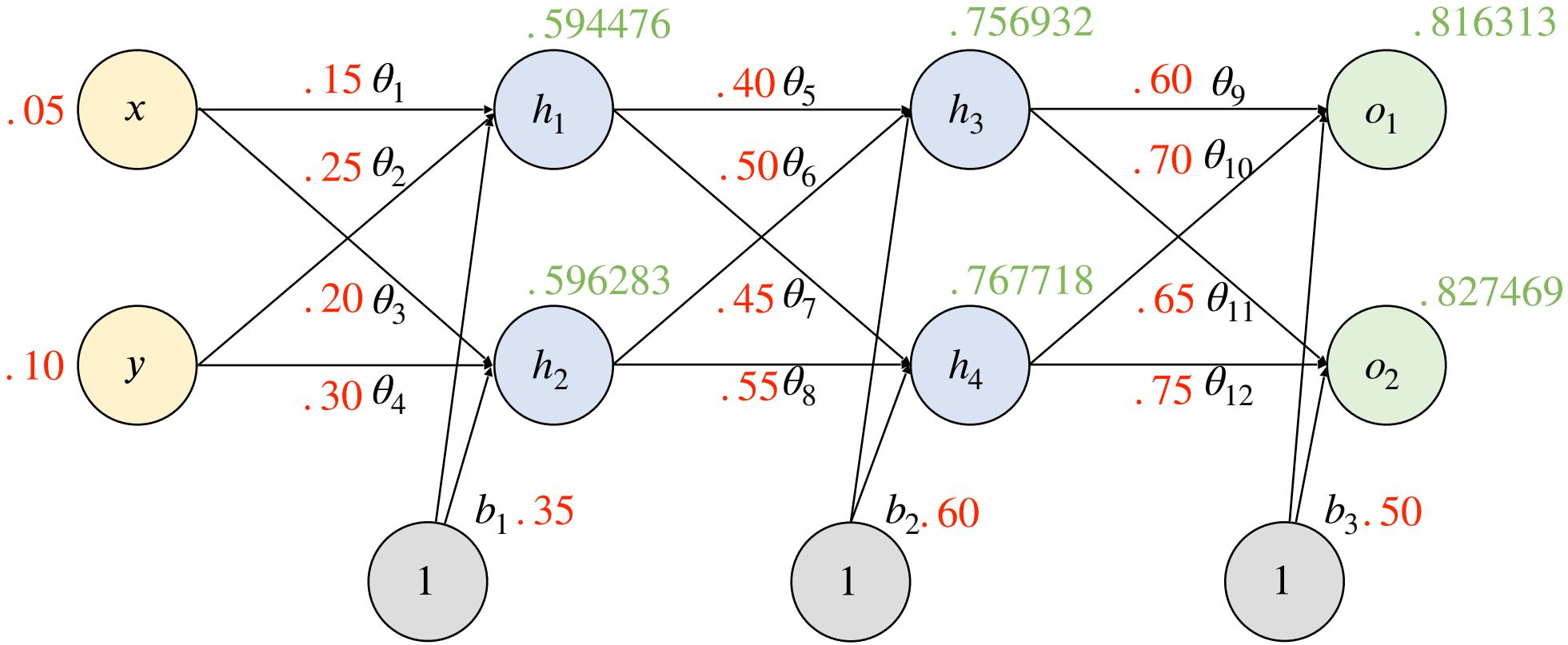
Simple Example 2 - Loss Calculation

Example 2:

$$Loss = \frac{1}{2}\left[(0.816313 - 0.01)^2 + (0.827469 - 0.99)^2\right] = 0.338278$$

Learning rate: 0.5 **Loss:** L2

Target: $(t1, t2) = (0.01, 0.99)$

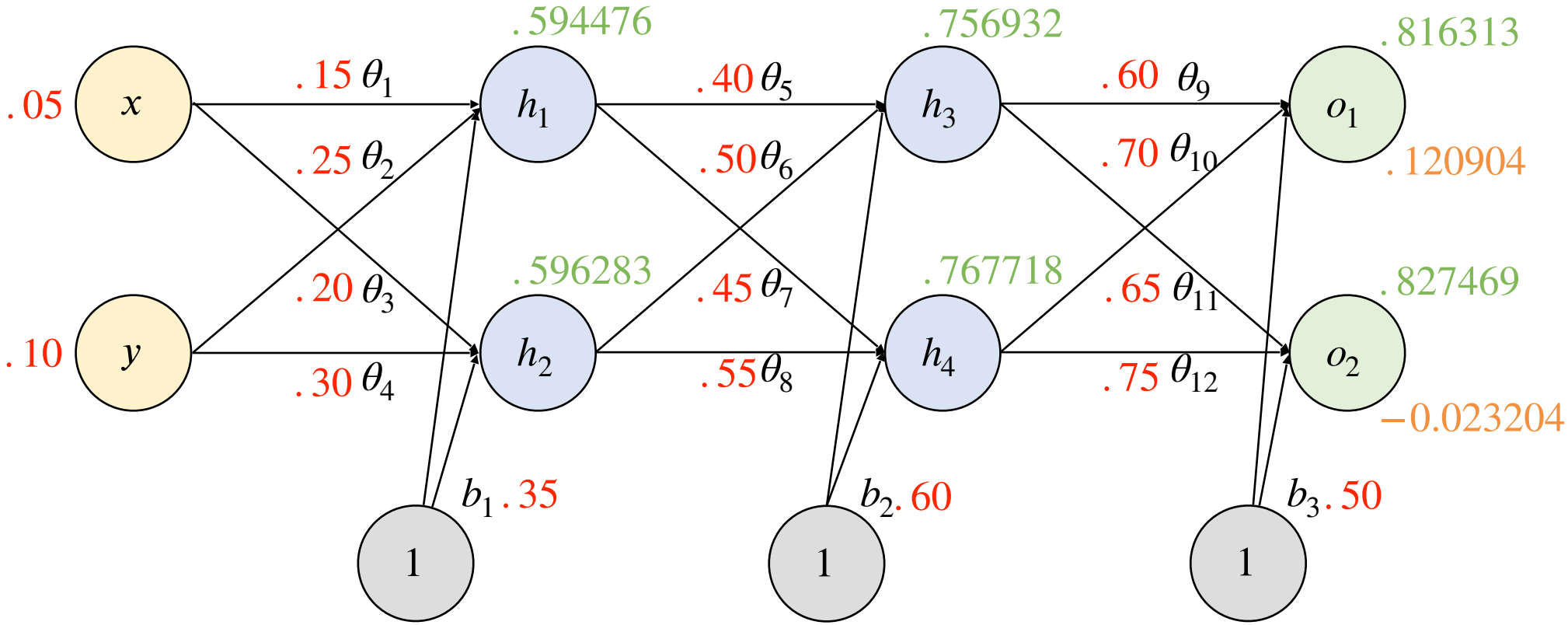


Backpropagation

Simple Example 2 - Backward Pass (Output Layer)

Example 2:

$$\delta_{o_1} = \underbrace{(o_1 - t_1)}_{\frac{\partial L}{\partial o_1}} \cdot \underbrace{o_1 \cdot (1 - o_1)}_{\frac{\partial o_1}{\partial z_{h_1}}} = 0.120904, \quad \delta_{o_2} = \underbrace{(o_2 - t_2)}_{\frac{\partial L}{\partial o_2}} \cdot \underbrace{o_2 \cdot (1 - o_2)}_{\frac{\partial o_2}{\partial z_{h_2}}} = -0.023204$$

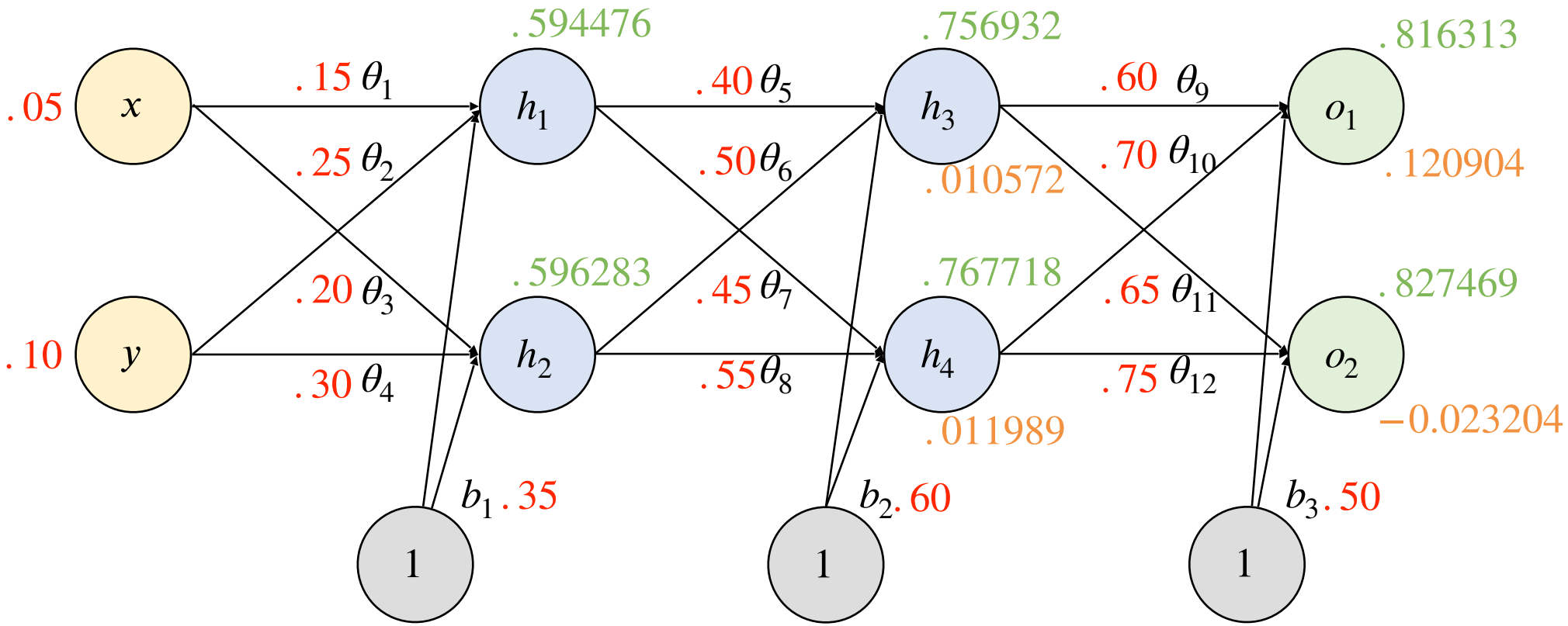


Backpropagation

Simple Example 2 - Backward Pass (Middle Layer)

Example 2:

$$\delta_{h_3} = \underbrace{(\theta_9 \cdot \delta_{o_1} + \theta_{11} \cdot \delta_{o_2})}_{\sum_k \theta_{3k} \cdot \delta_{o_k}} \cdot \underbrace{a_{h_3} \cdot (1 - a_{h_3})}_{\sigma'(z_{h_3})} = 0.010572, \quad \delta_{h_4} = \underbrace{(\theta_{10} \cdot \delta_{o_1} + \theta_{12} \cdot \delta_{o_2})}_{\sum_k \theta_{4k} \cdot \delta_{o_k}} \cdot \underbrace{a_{h_4} \cdot (1 - a_{h_4})}_{\sigma'(z_{h_4})} = 0.011989$$

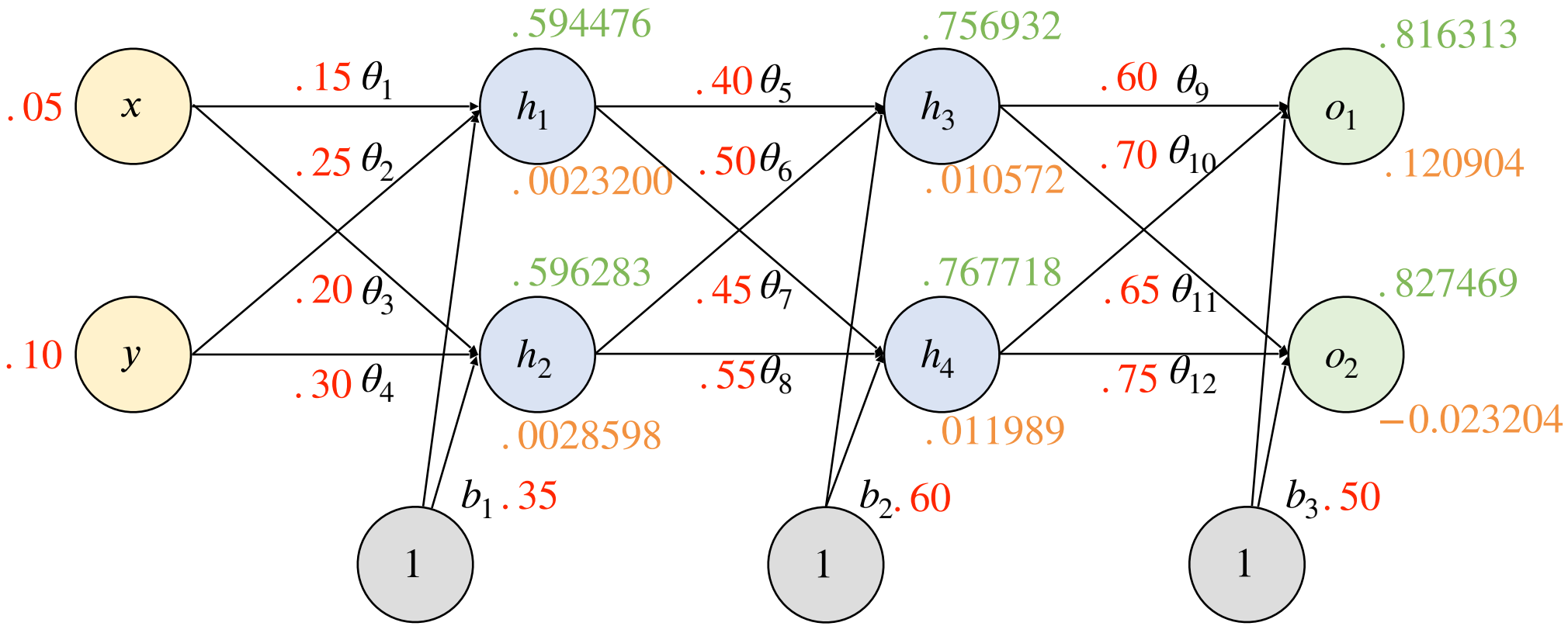


Backpropagation

Simple Example 2 - Backward Pass (Middle Layer)

Example 2:

$$\delta_{h_1} = \underbrace{(\theta_5 \cdot \delta_{h_3} + \theta_7 \cdot \delta_{h_4})}_{\sum_j \theta_{1j} \cdot \delta_{h_j}} \cdot \underbrace{a_{h_1} \cdot (1 - a_{h_1})}_{\sigma'(z_1)} = 0.0023200, \quad \delta_{h_2} = \underbrace{(\theta_6 \cdot \delta_{h_3} + \theta_8 \cdot \delta_{h_4})}_{\sum_j \theta_{2j} \cdot \delta_{h_j}} \cdot \underbrace{a_{h_2} \cdot (1 - a_{h_2})}_{\sigma'(z_2)} = 0.0028598$$

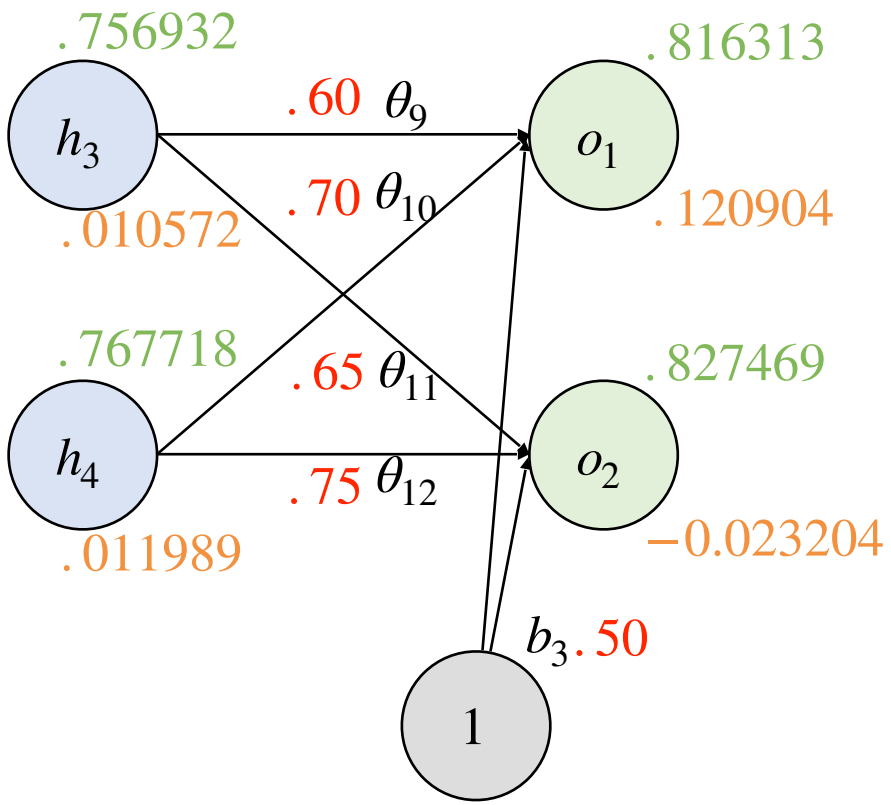


Backpropagation

Simple Example 2 - Update Parameters (Output Layer)

Example 2:

$$\frac{d\mathcal{L}}{d\theta_9} = a_{h_3}\delta_{o_1} = 0.091516$$
$$\frac{d\mathcal{L}}{d\theta_{10}} = a_{h_4}\delta_{o_1} = 0.092820$$
$$\frac{d\mathcal{L}}{d\theta_{11}} = a_{h_3}\delta_{o_2} = -0.017564$$
$$\frac{d\mathcal{L}}{d\theta_{12}} = a_{h_4}\delta_{o_2} = -0.017814$$
$$\frac{d\mathcal{L}}{db_3} = \delta_{o_1} + \delta_{o_2} = 0.097700$$



Update!

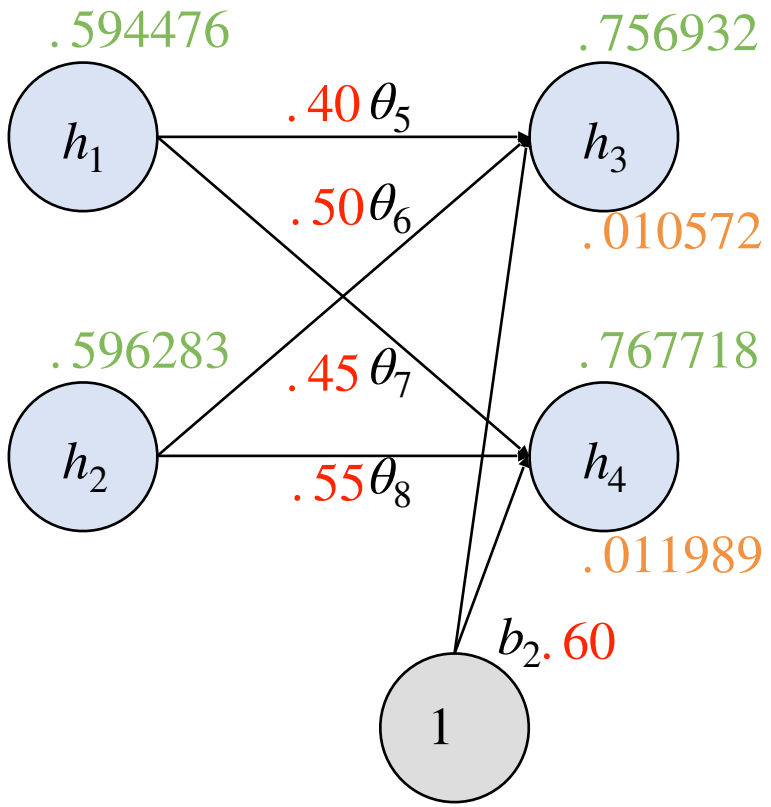
$$\theta_9 \leftarrow \theta_9 - 0.5 \cdot \frac{d\mathcal{L}}{d\theta_9} = 0.554242$$
$$\theta_{10} \leftarrow \theta_{10} - 0.5 \cdot \frac{d\mathcal{L}}{d\theta_{10}} = 0.653500$$
$$\theta_{11} \leftarrow \theta_{11} - 0.5 \cdot \frac{d\mathcal{L}}{d\theta_{11}} = 0.658782$$
$$\theta_{12} \leftarrow \theta_{12} - 0.5 \cdot \frac{d\mathcal{L}}{d\theta_{12}} = 0.758907$$
$$b_3 \leftarrow b_3 - 0.5 \cdot \frac{d\mathcal{L}}{db_3} = 0.451150$$

Backpropagation

Simple Example 2 - Update Parameters (Middle Layer)

Example 2:

$$\frac{d\mathscr{L}}{d\theta_5} = a_{h_1}\delta_{h_3} = 0.006285$$
$$\frac{d\mathscr{L}}{d\theta_6} = a_{h_2}\delta_{h_3} = 0.006304$$
$$\frac{d\mathscr{L}}{d\theta_7} = a_{h_1}\delta_{h_4} = 0.007127$$
$$\frac{d\mathscr{L}}{d\theta_8} = a_{h_2}\delta_{h_4} = 0.007149$$
$$\frac{d\mathscr{L}}{db_2} = \delta_{h_3} + \delta_{h_4} = 0.022561$$



Update!

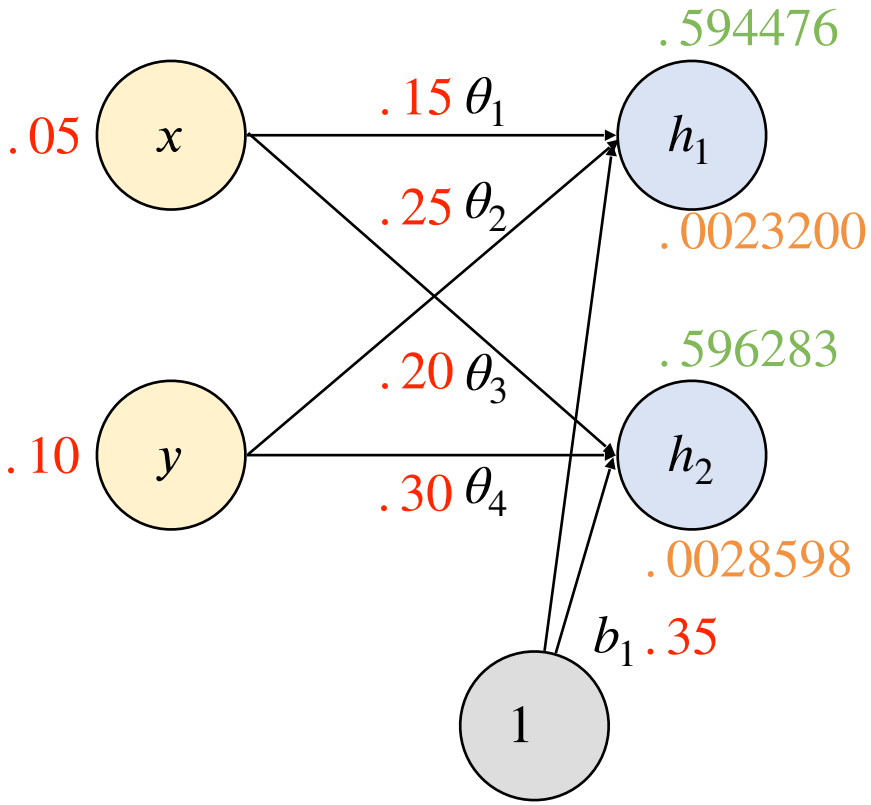
$$\theta_5 \leftarrow \theta_5 - 0.5 \cdot \frac{d\mathscr{L}}{d\theta_5} = 0.396858$$
$$\theta_6 \leftarrow \theta_6 - 0.5 \cdot \frac{d\mathscr{L}}{d\theta_6} = 0.496848$$
$$\theta_7 \leftarrow \theta_7 - 0.5 \cdot \frac{d\mathscr{L}}{d\theta_7} = 0.446436$$
$$\theta_8 \leftarrow \theta_8 - 0.5 \cdot \frac{d\mathscr{L}}{d\theta_8} = 0.546426$$
$$b_2 \leftarrow b_2 - 0.5 \cdot \frac{d\mathscr{L}}{db_2} = 0.588720$$

Backpropagation

Simple Example 2 - Update Parameters (Middle Layer)

Example 2:

$$\frac{d\mathscr{L}}{d\theta_1} = x\delta_{h_1} = 0.0001160$$
$$\frac{d\mathscr{L}}{d\theta_2} = y\delta_{h_1} = 0.0002320$$
$$\frac{d\mathscr{L}}{d\theta_3} = x\delta_{h_2} = 0.0001430$$
$$\frac{d\mathscr{L}}{d\theta_4} = y\delta_{h_2} = 0.0002860$$
$$\frac{d\mathscr{L}}{db_1} = \delta_{h_1} + \delta_{h_2} = 0.0051799$$



Update!

$$\theta_1 \leftarrow \theta_1 - 0.5 \cdot \frac{d\mathscr{L}}{d\theta_1} = 0.149942$$
$$\theta_2 \leftarrow \theta_2 - 0.5 \cdot \frac{d\mathscr{L}}{d\theta_2} = 0.249884$$
$$\theta_3 \leftarrow \theta_3 - 0.5 \cdot \frac{d\mathscr{L}}{d\theta_3} = 0.199929$$
$$\theta_4 \leftarrow \theta_4 - 0.5 \cdot \frac{d\mathscr{L}}{d\theta_4} = 0.299857$$
$$b_1 \leftarrow b_1 - 0.5 \cdot \frac{d\mathscr{L}}{db_1} = 0.347410$$

Backpropagation

What Makes Backpropagation Efficient?

- **Reuse / Memorization:** The forward pass caches h and activations a. In the backward pass each node's error signal δ is computed once, and each edge does a single multiply–add

  ⇒ time complexity *O(#edges)*, same order as the forward pass

- **Layer-wise vectorization:** $\delta_l = (\theta_l)^T \delta_{l+1} \odot a'(h_l)$

  (⊙ = element-wise product) – implemented with BLAS/GPU matrix ops for high throughput.

Backpropagation

Activation Function Summary - Useful Gradient Table

Activation Functions	Formula	Derivative	Tasks
Sigmoid	$\sigma(x) = \frac{1}{1+e^{-x}}$	$\sigma(x)(1-\sigma(x))$	Binary classification (logistic regression)
Tanh	$\tanh(x)$	$1-\tanh^2(x)$	RNN hidden layers
ReLU	$\max(0,\ x)$	$\begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x \leqslant 0 \end{cases}$	CNNs, hidden layers in deep neural networks
Leaky ReLU	$\max(0.1x,\ x)$	$\begin{cases} 1 & \text{if } x > 0 \\ 0.1 & \text{if } x \leqslant 0 \end{cases}$	CNNs (to avoid "dying ReLU" problem), generative models
Maxout	$\max(\theta_1^T x + b_1,\ \theta_2^T x + b_2)$	$\begin{cases} \theta_1, & \text{if } \theta_1^T x + b_1 > \theta_2^T x + b_2, \\ \theta_2, & \textit{otherwise}\ . \end{cases}$	Reinforcement learning, GANs (higher expressiveness)
ELU	$\begin{cases} x, & \text{if } x \geq 0, \\ \alpha(e^x - 1), & \text{if } x < 0, \end{cases}$	$\begin{cases} 1, & \text{if } x \geq 0, \\ \alpha e^x, & \text{if } x < 0. \end{cases}$	CNNs, deep neural networks (smooth alternative to ReLU, reduces bias shift)

Outline

- Neural Network
- Backpropagation
- **Training scheme**
- Deep neural network



Training Scheme

Training a Simple Neural Network Demonstration

```python
import numpy as np
from numpy.random import randn

N, Din, H, Dout = 64, 1000, 100, 10
x, y = randn(N, Din), randn(N, Dout)
w1, w2 = randn(Din, H), randn(H, Dout)
for t in range(10000):
    h = 1.0 / (1.0 + np.exp(-x.dot(w1)))
    y_pred = h.dot(w2)
    loss = np.square(y_pred - y).sum()
    dy_pred = 2.0 * (y_pred - y)
    dw2 = h.T.dot(dy_pred)
    dh = dy_pred.dot(w2.T)
    dw1 = x.T.dot(dh * h * (1 - h))
    w1 -= 1e-4 * dw1
    w2 -= 1e-4 * dw2
```

Initialize weights and data

Activation function: sigmoid

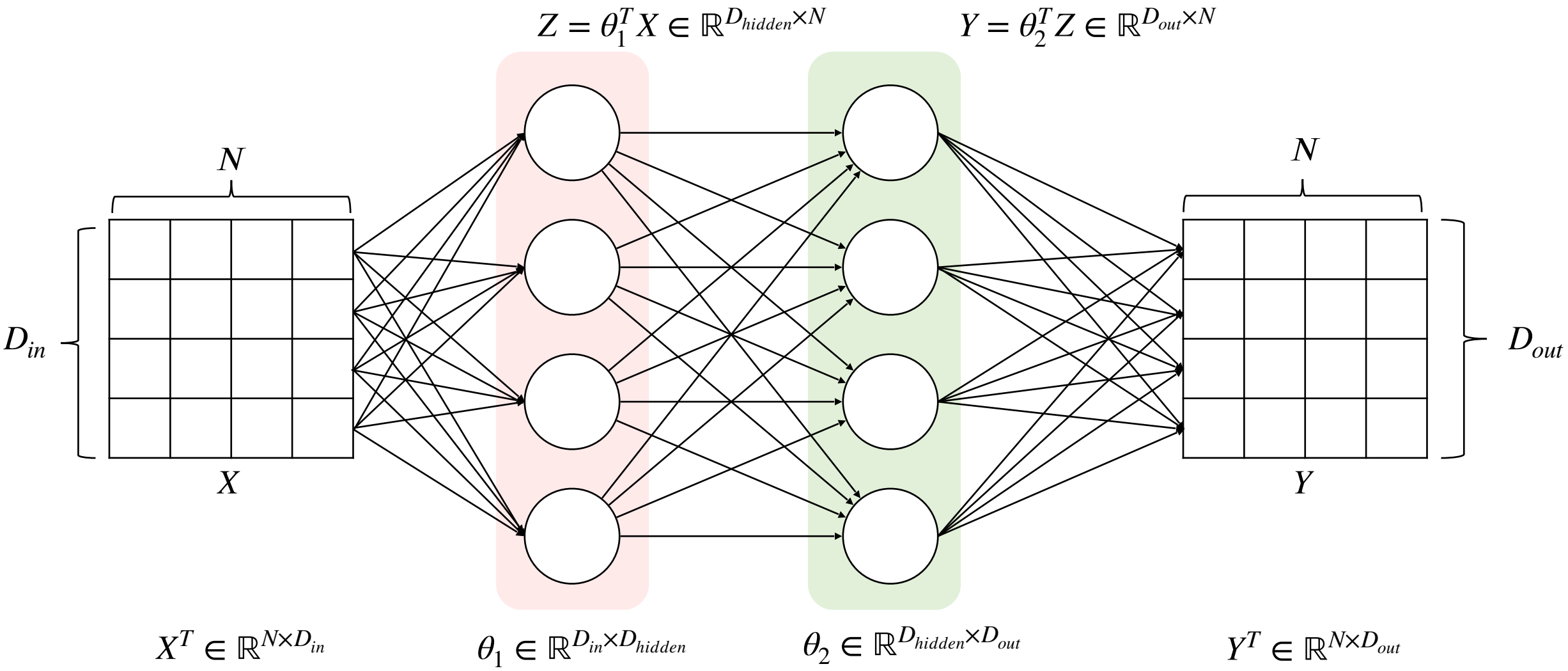
Compute $L2$ loss

Compute gradients

Update weights

Training Scheme

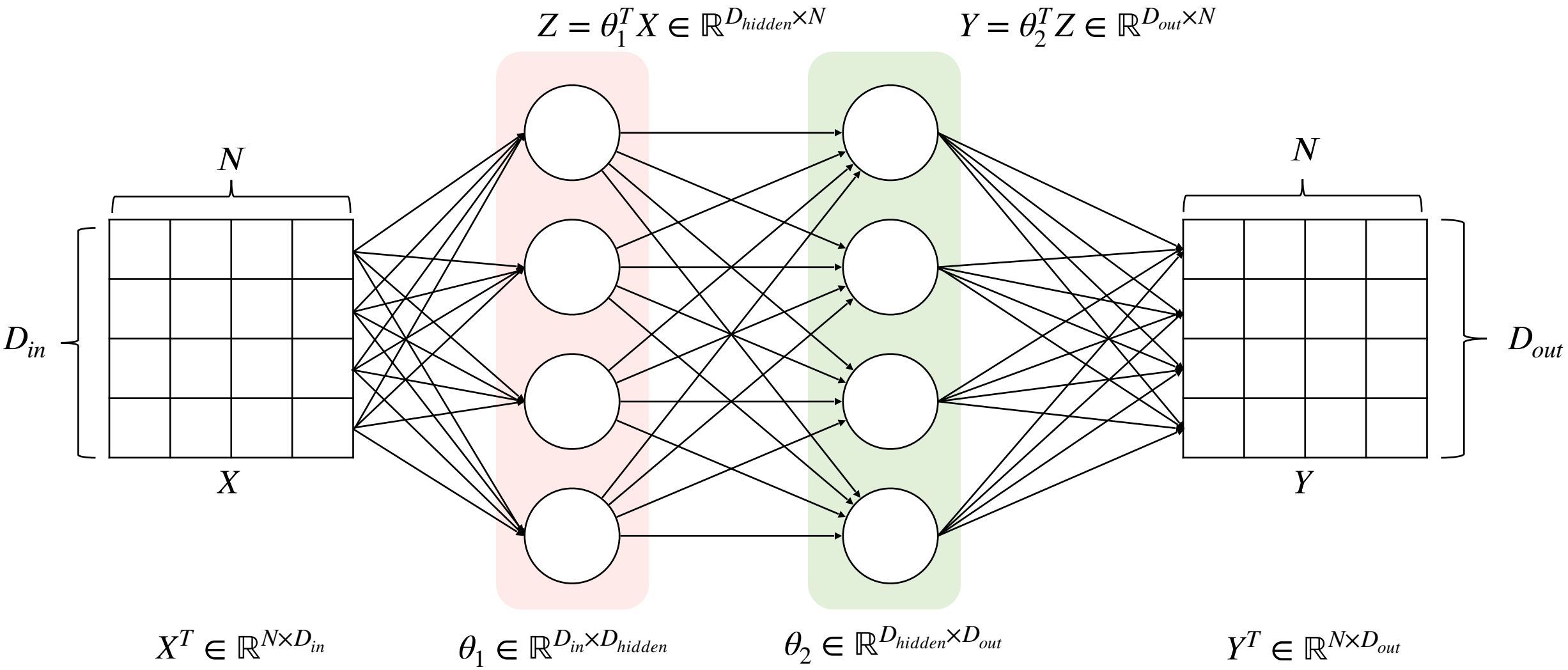
Neural Network Architecture



Training Scheme

Neural Network Architecture

Q: What happens if we initialize all $\theta = 0$?
A: All outputs are 0, all gradients are the same!
No "symmetry breaking"



Training Scheme

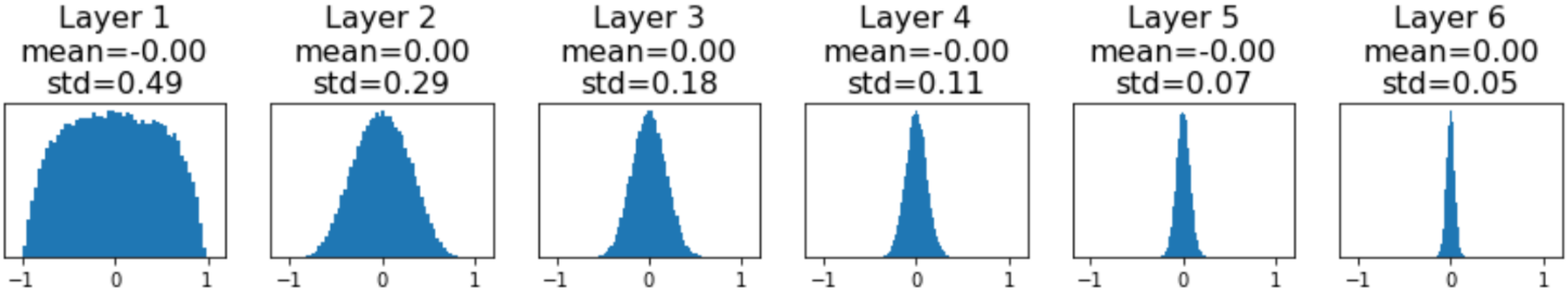
Randomly Initialized Weight

Questions: improper initial weights may cause training unsuccessful

Forward pass for a 6-layer net with hidden size 4096:

- Weights = small random numbers (Gaussian with mean = zero, std = 0.01)
- Activation Function: $\tanh(x)$

θ `= 0.01 * np.random.randn(Din, Dout)`



Gradients $d\mathscr{L}/d\theta$
→ All zero, no learning

Training Scheme

Randomly Initialized Weight

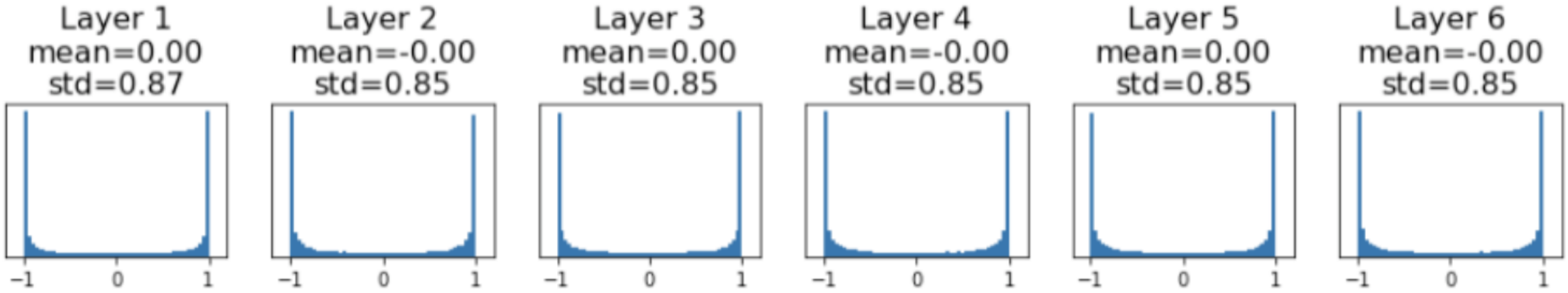
Questions: improper initial weights may cause training unsuccessful

Forward pass for a 6-layer net with hidden size 4096:

- Weights = small random numbers (Gaussian with mean = zero, std = 0.05)
- Activation Function = tanh(x)

Increase std from 0.01 → 0.05

θ `= 0.05 * np.random.randn(Din, Dout)`



Local gradients $d\mathcal{L}/d\theta$
→ All zero, no learning

Training Scheme

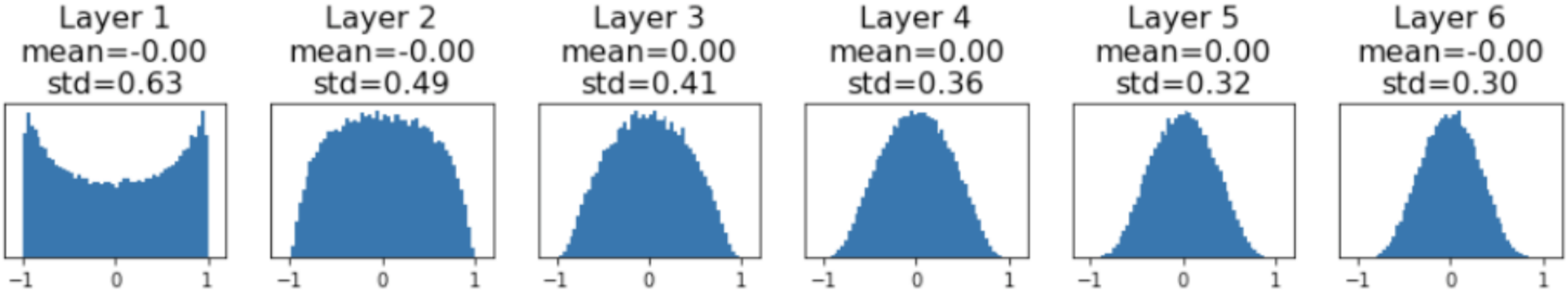
Randomly Initialized Weight - Xavier Initialization

Key: *Variance of output = Variance of input*

Forward pass for a 6-layer net with hidden size 4096:

- $std = 1/sqrt(Din)$
- Activation Function = tanh(x)

θ `= np.random.randn(Din, Dout) / np.sqrt(Din)`

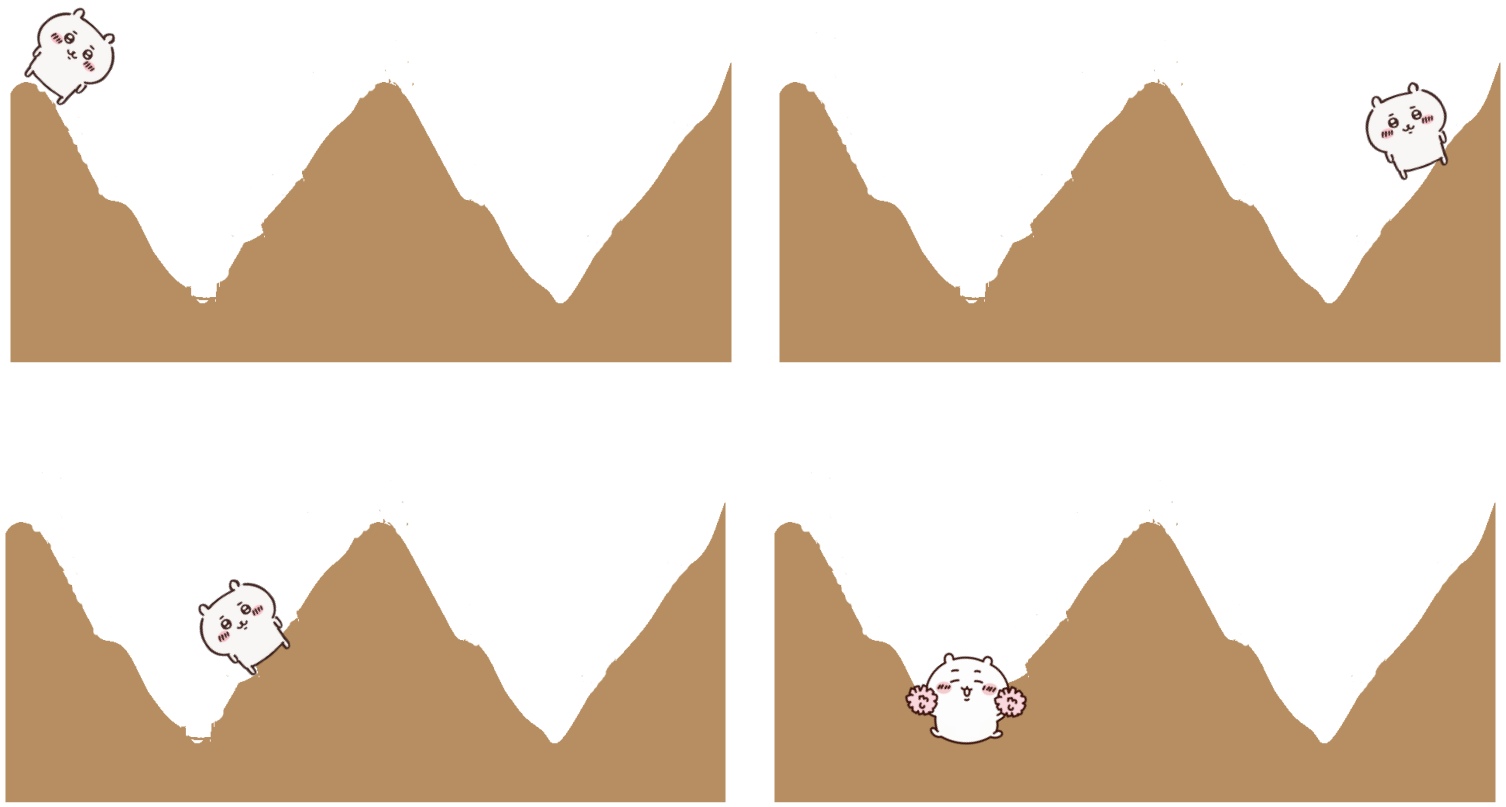


Activations are nicely scaled for all layers!

Training Scheme

Summary

If training fails…



Try different seeds and
You don't need ML!!
(nearly impossible :P)



Training Scheme

Gradient Descent (GD) v.s. Stochastic Gradient Descent (SGD)

- **Gradient descent update**
  - Use all training data to compute gradient each step
  - Stable convergence but can be slow (especially on large datasets)
  - Require more memory and computation per update
  - Move smoothly toward the minimum



- **Stochastic gradient descent**
  - Use one sample per update
  - Noisy updates lead to zig-zag path, which is less stable
  - Much faster per update and works well for large dataset

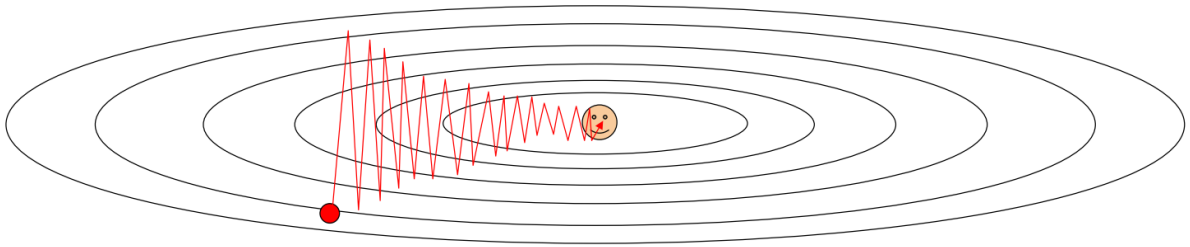


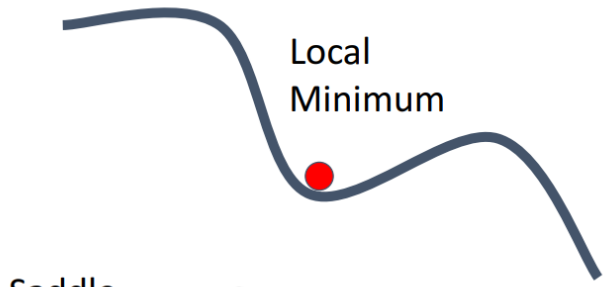
Training Scheme

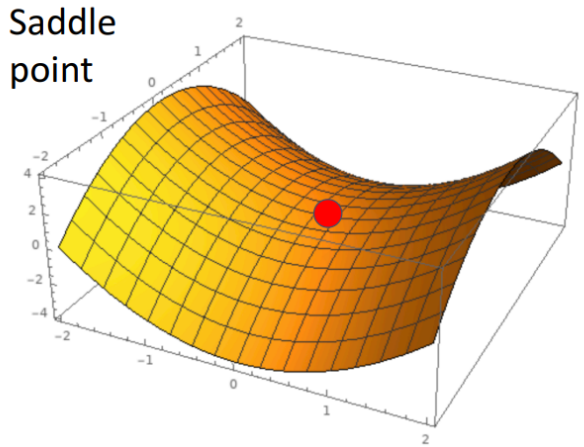
Limitations and Challenges of SGD

Problems with SGD:

- Sensitive to feature scaling
- Noisy updates
- Getting stuck in local minima or saddle points







Training Scheme

Mini-batch Stochastic Gradient Descent (Mini-Batch SGD)

- GD and SGD are extreme scenarios
- Computation time per update does not depend on the number of training examples N
- This allows convergence on extremely large datasets

Stochastic Gradient Descent (SGD) update at training iteration k

Require: Learning rate η

Require: Initial parameter θ

while stopping criterion not met **do**

  Sample a mini-batch of m examples from the training set $\{(x_i, y_i)\}_{i=1}^{N}$

  Compute gradient estimate: $\hat{g} \leftarrow \frac{1}{m} \nabla_\theta \sum_i L\big(f(x_i; \theta), y_i\big)$

  Apply update: $\theta \leftarrow \theta - \eta \hat{g}$

end while

Training Scheme

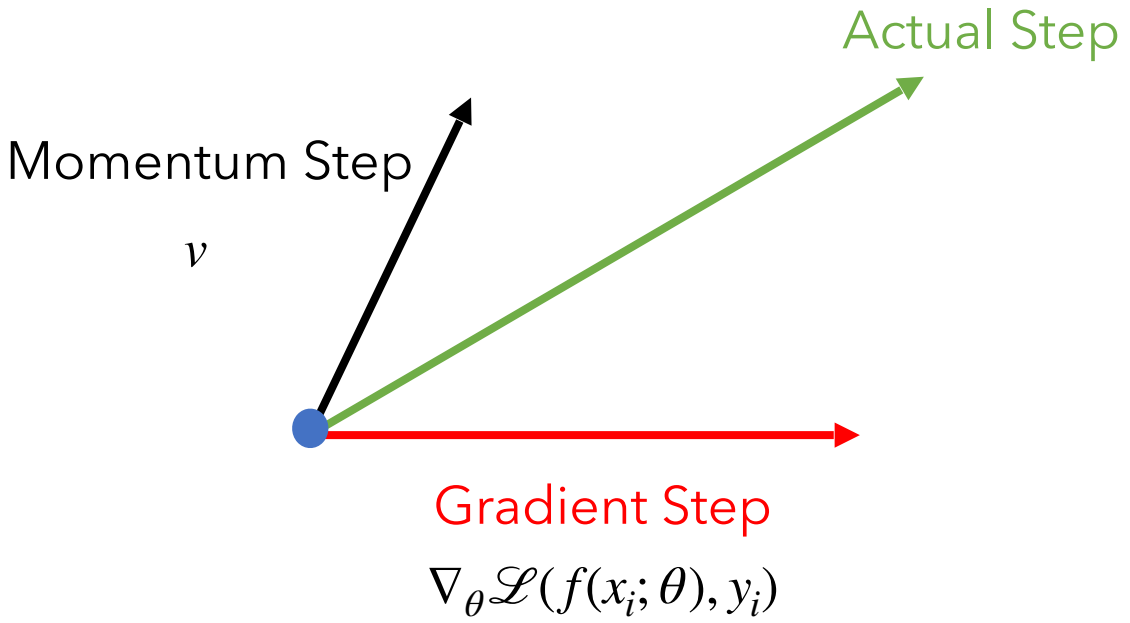
Momentum

Momentum *is a method to accelerate learning using SGD*

- Introduce a new variable v, the velocity
- Update rule:

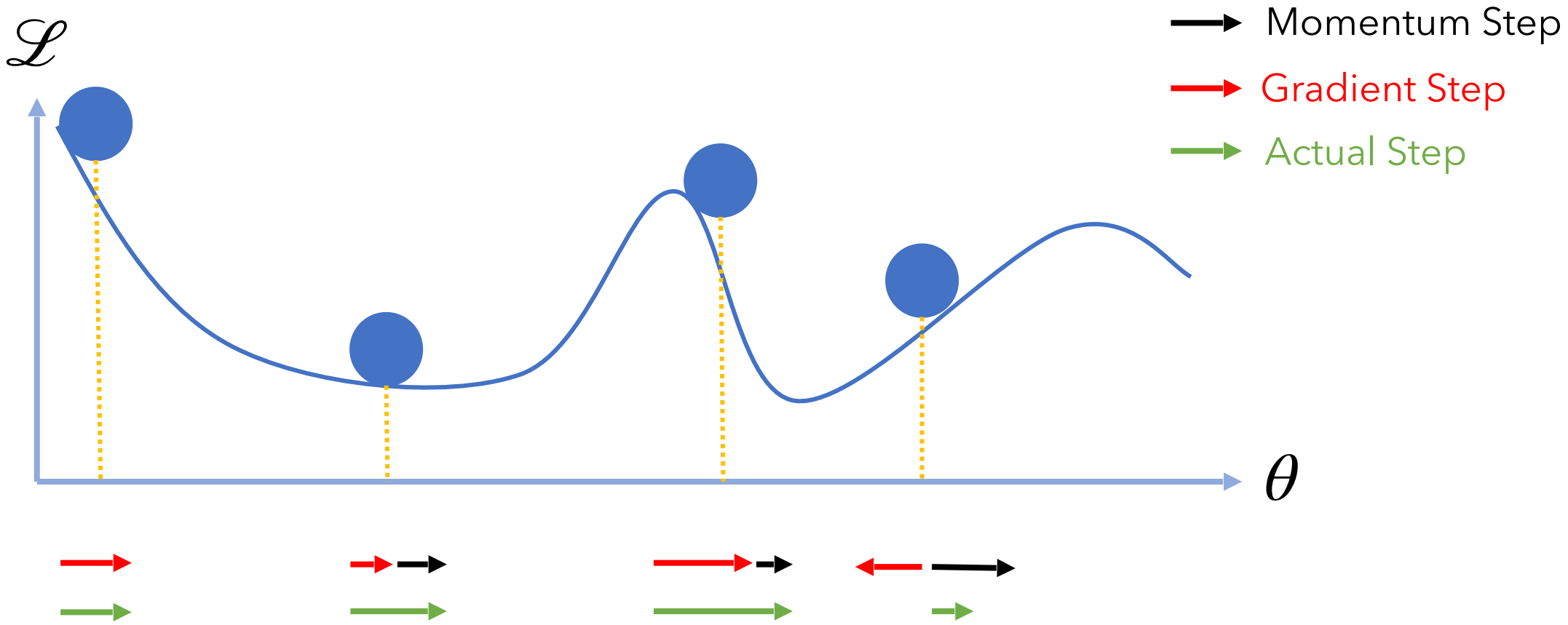
$$v \leftarrow \alpha v - \eta \nabla_\theta \mathscr{L}(f(x_i;\theta), y_i)$$

$$\theta \leftarrow \theta + v$$



Training Scheme

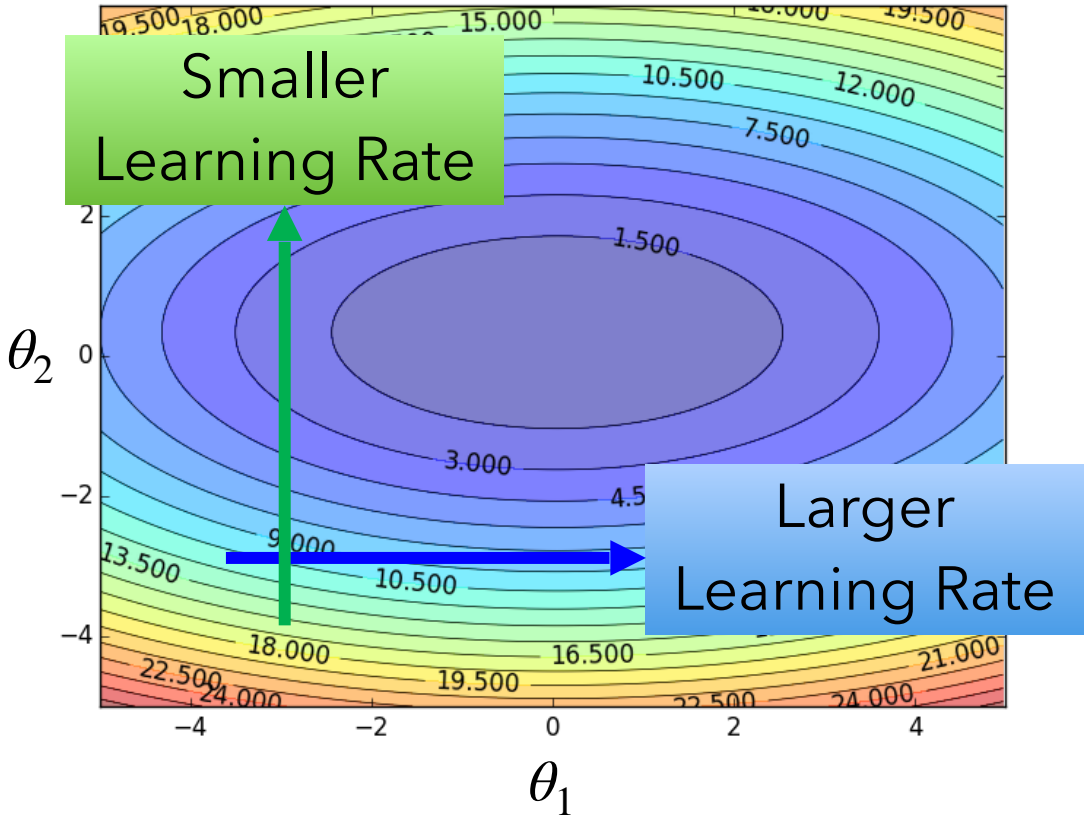
Momentum

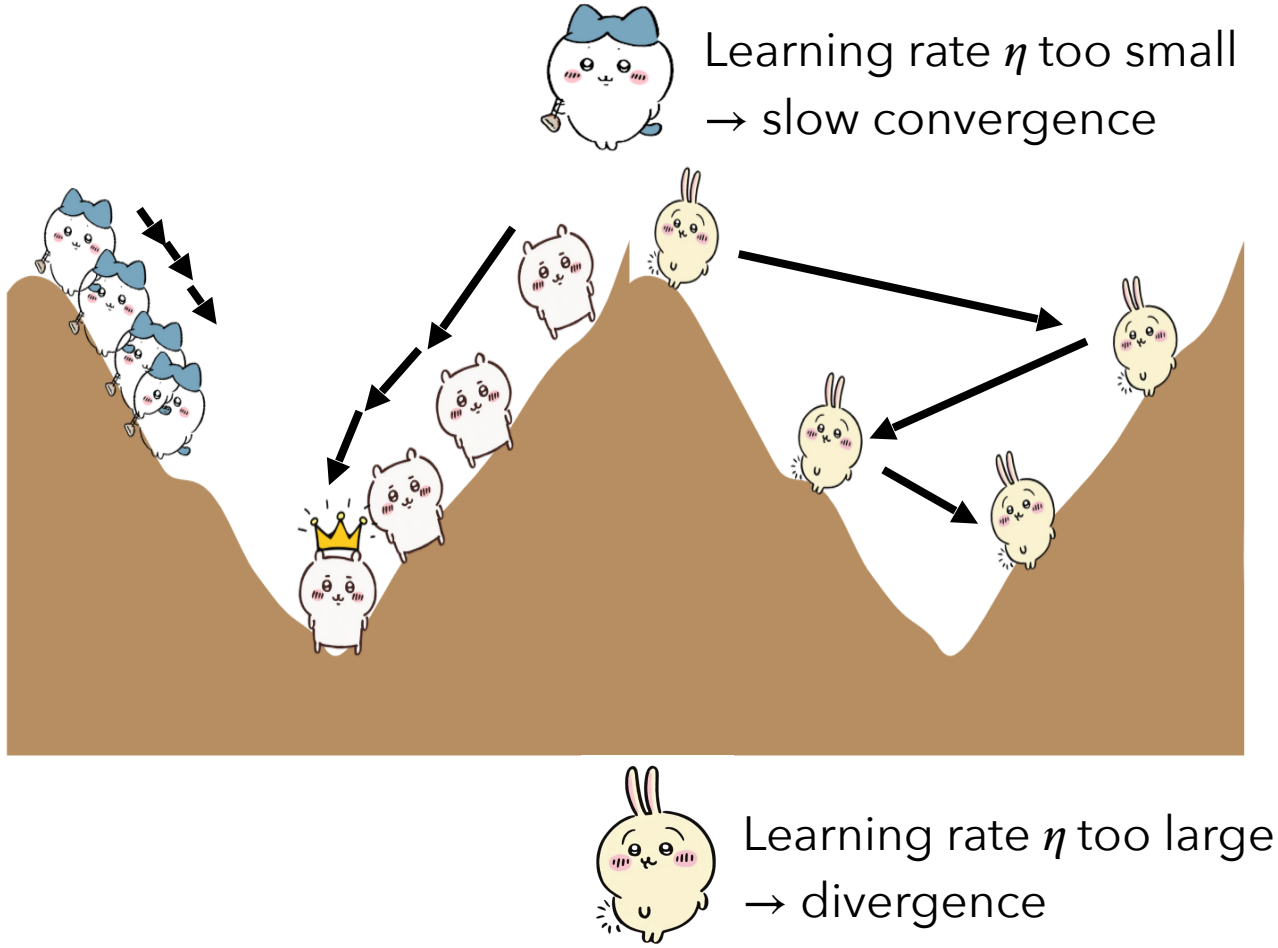


Training Scheme

Adaptive Learning Rate Method

Idea: If the features vary in importance and frequency, using same learning rate may not be a good idea





Training Scheme

Adaptive Learning Rate Method - AdaGrad

AdaGrad automatically **adapts the learning rate for each parameter** based on the history of gradients for that parameter

Idea: Parameters that receive large gradients frequently should have their learning rate decreased, while parameters with small gradients should have their learning rate increased

Algorithm AdaGrad (Adaptive Gradient)

Initialize parameters $\theta_0 \in \mathbb{R}^d$, learning rate $\eta > 0$, $\epsilon = 10^{-8}$
Initialize gradient accumulator $G_0 = 0 \in \mathbb{R}^d$
for $t = 1$ to T **do**
  Compute gradient: $g_t = \nabla_\theta \mathcal{L}(\theta_{t-1})$
  Accumulate squared gradients: $G_t = G_{t-1} + g_t \odot g_t$
  Update parameters: $\theta_t = \theta_{t-1} - \dfrac{\eta}{\sqrt{G_t + \epsilon}} \odot g_t$
end for
return θ_T

Training Scheme

RMSProp (Root Mean Square Propagation)

Key Idea: Uses exponential moving average of squared gradients instead of cumulative sum to prevent learning rate from decaying too aggressively

Algorithm RMSProp (Root Mean Square Propagation)

Initialize parameters $\theta_0 \in \mathbb{R}^d$, learning rate $\eta > 0$, decay rate $\beta = 0.9$, $\epsilon = 10^{-8}$
Initialize moving average $v_0 = 0 \in \mathbb{R}^d$
for $t = 1$ to T **do**
  Compute gradient: $g_t = \nabla_\theta \mathcal{L}(\theta_{t-1})$
  Update moving average: $v_t = \beta v_{t-1} + (1 - \beta) g_t \odot g_t$
  Update parameters: $\theta_t = \theta_{t-1} - \dfrac{\eta}{\sqrt{v_t + \epsilon}} \odot g_t$
end for
return θ_T

Training Scheme

Adam (Adaptive Moment Estimation)

Key Idea: Combines momentum (first moment) and RMSProp (second moment) with bias correction for better convergence

Algorithm Adam (Adaptive Moment Estimation)

Initialize parameters $\theta_0 \in \mathbb{R}^d$, learning rate $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$
Initialize first moment $m_0 = 0 \in \mathbb{R}^d$ and second moment $v_0 = 0 \in \mathbb{R}^d$
for $t = 1$ to T **do**
  Compute gradient: $g_t = \nabla_\theta \mathcal{L}(\theta_{t-1})$
  Update biased first moment: $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$
  Update biased second moment: $v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t \odot g_t$
  Correct first moment bias: $\hat{m}_t = \dfrac{m_t}{1 - \beta_1^t}$
  Correct second moment bias: $\hat{v}_t = \dfrac{v_t}{1 - \beta_2^t}$
  Update parameters: $\theta_t = \theta_{t-1} - \dfrac{\eta}{\sqrt{\hat{v}_t + \epsilon}} \odot \hat{m}_t$
end for
return θ_T

Regularization

Practical Tools

Regularization uses additional terms in training optimization objects to prevent overfitting or help the optimization.

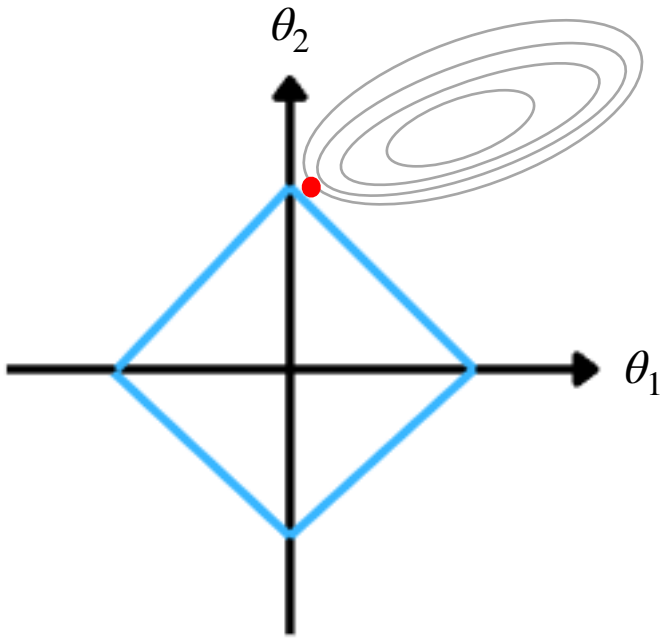
Classical regularization:

- **L1 regularization**

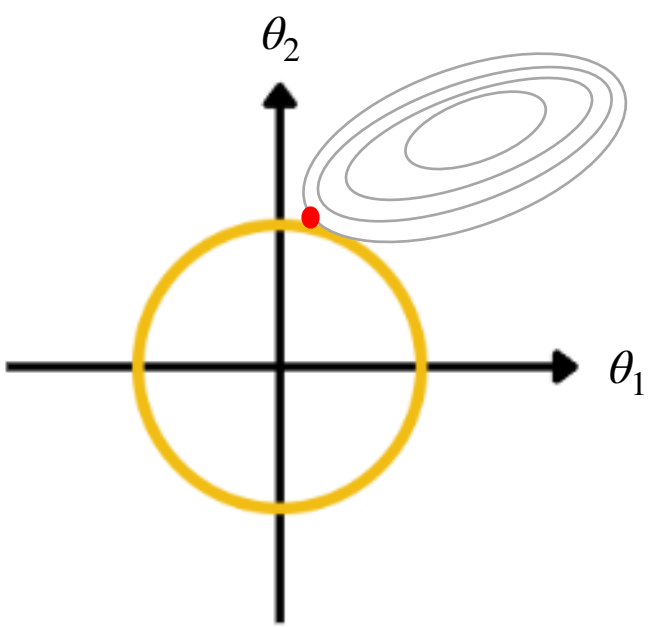
$$\min_{\theta} \sum_{i=1}^{n} \mathscr{L}\left(f(x_i), y_i\right) + \lambda \sum_{j=1}^{p} \left|\theta_j\right|$$

- **L2 regularization**

$$\min_{\theta} \sum_{i=1}^{n} \mathscr{L}\left(f(x_i), y_i\right) + \lambda \sum_{j=1}^{p} \theta_j^2$$



L1 Regularization



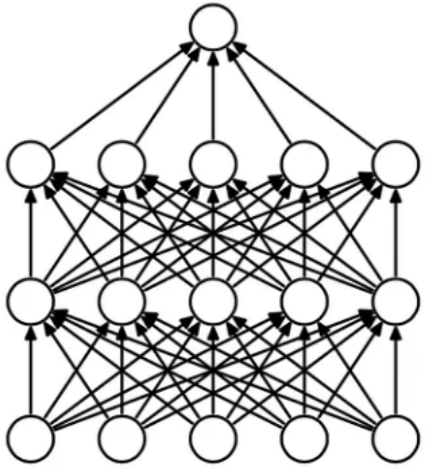
L2 Regularization

Dropout

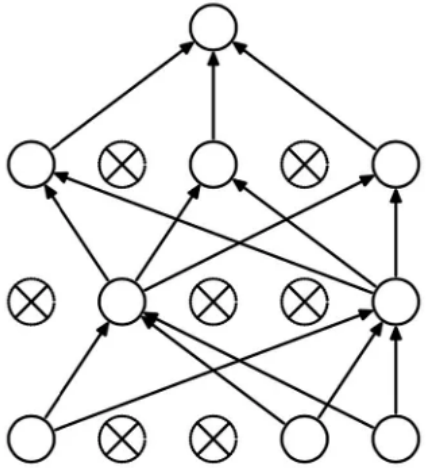
Practical Tools

Dropout is a regularization method: without dropout the model tends to memorize training data, while with dropout the model learns to generalize better

- Randomly drops out (sets to 0) a fraction of neurons during training
- Each hidden node has a probability p to be kept: creates thinned sub-networks sharing weights
- At test time, use the full network with scaled weights (no dropout applied)
- Improves generalization and reduces overfitting



(a) Standard Neural Net



(b) After applying dropout.



Normalization

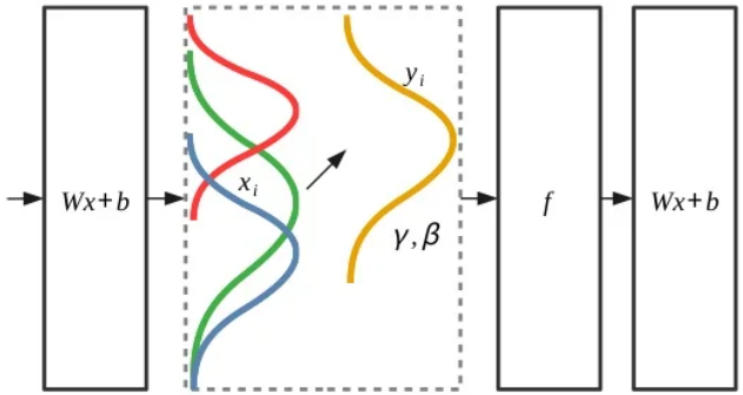
Practical Tools

- Reduces internal covariate shift (distributions of inputs to layers keep changing)
- Allows higher learning rates

  Less covariate shift, less exploding/vanishing gradients
- Adds noise during training (similar to dropout), reduces overfitting

  Means and Variances are calculated over batches, and therefore every normalized value depends on the current batch
- Enables training with saturating nonlinearities in deep networks, e.g., sigmoid

  Because the normalization prevents them from getting stuck in saturating ranges



Normalization

Batch Normalization (BN)

- To normalize the layers' inputs by re-centering and re-scaling
- $\mathscr{B}$: mini-batch of size m

Batch Normalizing Transform, applied to activation x over a mini-batch

Require: Values of x over a mini-batch: $\mathcal{B} = \{x_1, \ldots, x_m\}$
Require: Parameters to be learned: γ, β
Ensure: $\{y_i = BN_{\gamma,\beta}(x_i)\}$

1: Compute mini-batch mean:

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i$$

2: Compute mini-batch variance:

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2$$

3: Normalize:

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}$$

4: Scale and shift:

$$y_i \leftarrow \gamma \hat{x}_i + \beta \quad \equiv \quad BN_{\gamma,\beta}(x_i)$$

Outline

- Neural Network
- Backpropagation
- Training scheme
- **Deep neural network**



Shallow NN v.s. Deep NN

Different Numbers of Hidden Layers

- Definition:

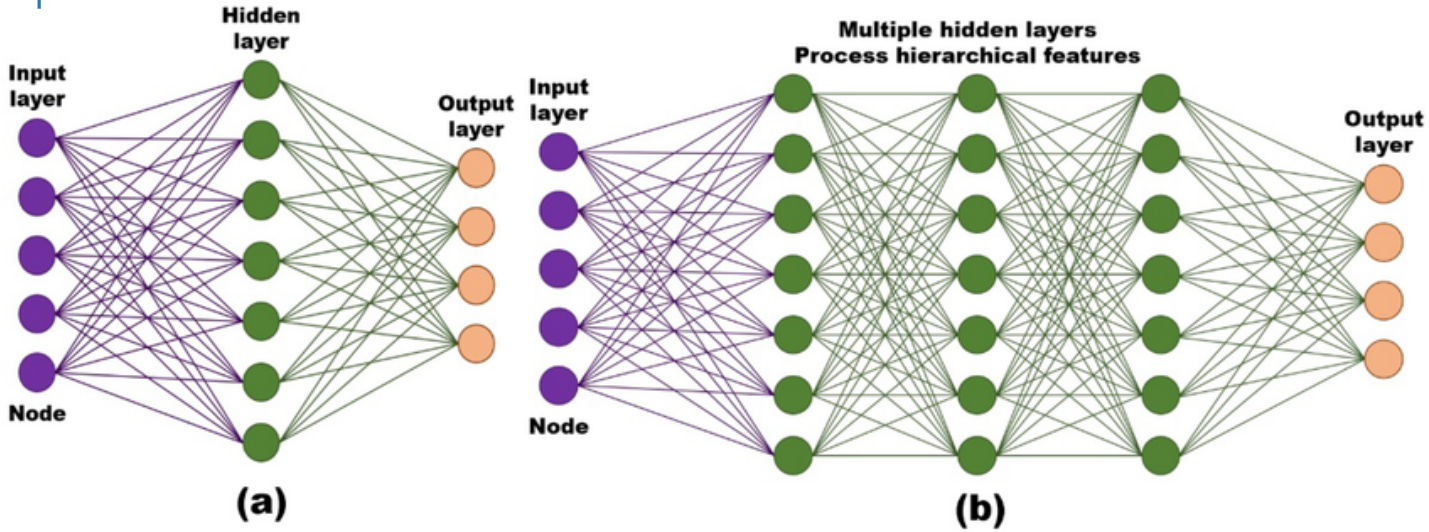
  Shallow = 1–2 hidden layers; Deep = 3+ hidden layers (often tens in practice)

- Why deep?

  Stacks of linear + nonlinearity learn hierarchical and can represent complex functions more compactly

- Trade-offs:

  Higher capacity but harder to optimize (vanishing/exploding gradients/…), overfitting risk, and more data/compute required.



Shallow NN v.s. Deep NN

Comparison of Shallow NN and Deep NN

Shallow NN	Deep NN
Shallow Neural network with few layers	Deep Neural network with many layers
Complexity is low	Complexity is high
Limited learning capacity	Higher learning capacity
Lower risk of overfitting	Higher risk of overfitting
Requires less data	Requires more data for effective training
Fewer parameters	Many more parameters
Requires less computational resources	Requires more computational resources (e.g., GPUs)
Example: Single-layer Perceptron, Logistic Regression	Example: Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs)

Thank You for Your Attention!



